DELHI TECHNOLOGICAL UNIVERSITY

(Formerly Delhi College of Engineering)

Shahbad Daulatpur, Bawana Road, Delhi 110042

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

DEEP LEARNING

LAB FILE

Subject Code: CS-318
E3(G2)

Submitted To:
Dr. Anukriti Kaushal
Assistant Professor
Dept. of CSE

Submitted By:
Pratham Jain
B.Tech
Roll no : 23/CS/311

INDEX

S. No	EXPERIMENT	DATE	SIGN
1	Implementation of an Artificial Neural Network Using Backpropagation with Varying Activation Functions.		
2	Implementation of a 1D Convolutional Neural Network for Sequence-Based Prediction Tasks.		
3	Design and Implementation of a Deep CNN with Padding and Batch Normalization for Multi-Class Image Classification.		
4	Design and Implementation of a CNN Model for Text Classification		
5	Implementation of Standard VGG-16 and VGG-19 Architectures for Multi-Class Image Classification.		
6	Implementation of a Recurrent Neural Network (RNN) for Sequential Data Analysis.		
7	Sentiment Analysis Using a Recurrent Neural Network (RNN) on Movie Review Data.		
8	Implementation of an LSTM Neural Network for Sequential Data Processing.		
9	Sentiment Analysis Using a Bidirectional LSTM on Movie Review Data.		
10	Implementation of a GRU Neural Network for Sequential Data Processing.		

Experiment 1

Aim - Implementation of an Artificial Neural Network Using Backpropagation with Varying Activation Functions

Theory -

An Artificial Neural Network is a computational model inspired by the biological neural network of the human brain. It consists of layers of neurons:

- **Input Layer**: Accepts input data
- **Hidden Layer(s)**: Performs computations
- **Output Layer**: Produces the final output

Each neuron computes a weighted sum of inputs and applies an activation function.

$$z = \sum w_i x_i + b$$

$$a = f(z)$$

The network is trained using the **backpropagation algorithm**, which minimizes the error between predicted and actual output. The process involves forward propagation to compute output, followed by backward propagation to update weights using gradient descent:

$$w = w - \eta \frac{\partial E}{\partial w}$$

where η is the learning rate.

Activation functions introduce non-linearity and improve learning capability. Common functions include:

- **Sigmoid**: Output between 0 and 1
- **Tanh**: Output between -1 and 1
- **ReLU**: Outputs 0 for negative and linear for positive values

Different activation functions affect the training speed and accuracy of the network. ReLU generally provides faster convergence, while Sigmoid and Tanh may suffer from vanishing gradient problems.

Code:

```
import torch
import torch.nn as nn
import torch.optim as
optim
from torchvision import datasets, transforms
from torch.utils.data import DataLoader, random_split
import matplotlib.pyplot as plt
import os

# Dataset & Dataloader

transform = transforms.Compose([
  transforms.ToTensor(),
  transforms.Normalize((0.1307,), (0.3081,))
])
dataset = datasets.MNIST(root="./data", train=True, download=True, transform=transform)
train_size = int(0.9 * len(dataset))
val_size = len(dataset) - train_size
train_ds, val_ds = random_split(dataset, [train_size, val_size])

train_loader = DataLoader(train_ds, batch_size=64, shuffle=True)
val_loader = DataLoader(val_ds, batch_size=64)


# Training Function

def train(model, optimizer, epochs=10):
  criterion = nn.CrossEntropyLoss()

  train_loss, val_loss = [], []
  train_acc, val_acc = [], []

  for epoch in range(epochs):
     model.train()
     correct, total, running_loss = 0, 0, 0
```

```
        for x, y in train_loader:
            x = x.view(x.size(0), -1)

            optimizer.zero_grad()
            out = model(x)
            loss = criterion(out, y)
            loss.backward()
            optimizer.step()

            running_loss += loss.item()
            correct += (out.argmax(1) == y).sum().item()
            total += y.size(0)

        train_loss.append(running_loss / len(train_loader))
        train_acc.append(correct / total)

        model.eval()
        correct, total, running_loss = 0, 0, 0
        with torch.no_grad():
            for x, y in val_loader:
                x = x.view(x.size(0), -1)
                out = model(x)
                loss = criterion(out, y)

                running_loss += loss.item()
                correct += (out.argmax(1) == y).sum().item()
                total += y.size(0)

        val_loss.append(running_loss / len(val_loader))
        val_acc.append(correct / total)

        print(f"Epoch {epoch+1:02d} | Train Acc: {train_acc[-1]:.4f} | Val Acc: {val_acc[-1]:.4f}")

    return train_loss, val_loss, train_acc, val_acc


# Model 1: Sigmoid + SGD
```

```
print("\n=== SIGMOID + SGD ===")

model_sigmoid = nn.Sequential(
  nn.Linear(784, 256),
  nn.Sigmoid(),
  nn.Linear(256, 128),
  nn.Sigmoid(),
  nn.Linear(128, 10)
)

optimizer_sgd = optim.SGD(model_sigmoid.parameters(), lr=0.1)
sigmoid_results = train(model_sigmoid, optimizer_sgd)


# Model 2: ReLU + Adam
print("\n=== ReLU + Adam (No Regularization) ===")
model_relu = nn.Sequential(
  nn.Linear(784, 256),
  nn.ReLU(),
  nn.Linear(256, 128),
  nn.ReLU(),
  nn.Linear(128, 10)
)

optimizer_adam = optim.Adam(model_relu.parameters(), lr=0.001)
relu_results = train(model_relu, optimizer_adam)


# ReLU + Adam + L2
print("\n=== ReLU + Adam + L2 ===")
model_l2 = nn.Sequential(
  nn.Linear(784, 256),
  nn.ReLU(),
  nn.Linear(256, 128),
  nn.ReLU(),
```

```
    nn.Linear(128, 10)
)

optimizer_l2 = optim.Adam(model_l2.parameters(), lr=0.001, weight_decay=1e-4)
l2_results = train(model_l2, optimizer_l2)


# ReLU + Adam + Dropout
print("\n=== ReLU + Adam + Dropout ===")
model_dropout = nn.Sequential(
    nn.Linear(784, 256),
    nn.ReLU(),
    nn.Dropout(0.5),
    nn.Linear(256, 128),
    nn.ReLU(),
    nn.Linear(128, 10)
)

optimizer_dropout = optim.Adam(model_dropout.parameters(), lr=0.001)
dropout_results = train(model_dropout, optimizer_dropout)


# Final Loss Printing

def print_final_losses(name,
    results): print(f"{name}")
    print(f" Final Training Loss : {results[0][-1]:.4f}")
    print(f" Final Validation Loss : {results[1][-1]:.4f}")
    print("-" * 40)

print("\n=== FINAL LOSSES SUMMARY ===")
print_final_losses("Sigmoid + SGD", sigmoid_results)
print_final_losses("ReLU + Adam (No Reg)", relu_results)
print_final_losses("ReLU + Adam + L2", l2_results)
print_final_losses("ReLU + Adam + Dropout", dropout_results)
# Plotting
os.makedirs("plots", exist_ok=True)
```

```
def plot_compare(curves, labels, title, ylabel, filename):
  plt.figure()
  for curve, label in zip(curves, labels):
    plt.plot(curve, label=label)
  plt.xlabel("Epochs")
  plt.ylabel(ylabel)
  plt.title(title)
  plt.legend()
  plt.tight_layout()
  plt.savefig(filename)
  plt.close()
plot_compare(
  [relu_results[0], l2_results[0], dropout_results[0]],
  ["No Reg", "L2", "Dropout"],
  "Training Loss vs Epochs",
  "Loss",
  "plots/train_loss.png"
)
plot_compare(
  [relu_results[1], l2_results[1], dropout_results[1]],
  ["No Reg", "L2", "Dropout"],
  "Validation Loss vs Epochs",
  "Loss",
  "plots/val_loss.png"
)
plot_compare(
  [relu_results[2], l2_results[2], dropout_results[2]],
  ["No Reg", "L2", "Dropout"],
  "Training Accuracy vs Epochs",
  "Accuracy",
  "plots/train_accuracy.png"
)

plot_compare(
  [relu_results[3], l2_results[3], dropout_results[3]],
  ["No Reg", "L2", "Dropout"],
  "Validation Accuracy vs Epochs",
  "Accuracy",
  "plots/val_accuracy.png"
```

```
)

print("\nPlots saved in ./plots/")
```

Output :

```
=== SIGMOID + SGD ===
Epoch 01 | Train Acc: 0.6548 | Val Acc: 0.8730
Epoch 02 | Train Acc: 0.8966 | Val Acc: 0.9073
Epoch 03 | Train Acc: 0.9179 | Val Acc: 0.9218
Epoch 04 | Train Acc: 0.9323 | Val Acc: 0.9327
Epoch 05 | Train Acc: 0.9432 | Val Acc: 0.9432
Epoch 06 | Train Acc: 0.9518 | Val Acc: 0.9505
Epoch 07 | Train Acc: 0.9588 | Val Acc: 0.9545
Epoch 08 | Train Acc: 0.9641 | Val Acc: 0.9622
Epoch 09 | Train Acc: 0.9687 | Val Acc: 0.9623
Epoch 10 | Train Acc: 0.9727 | Val Acc: 0.9658

=== ReLU + Adam (No Regularization) ===
Epoch 01 | Train Acc: 0.9251 | Val Acc: 0.9613
Epoch 02 | Train Acc: 0.9685 | Val Acc: 0.9738
Epoch 03 | Train Acc: 0.9788 | Val Acc: 0.9732
Epoch 04 | Train Acc: 0.9834 | Val Acc: 0.9770
Epoch 05 | Train Acc: 0.9864 | Val Acc: 0.9752
Epoch 06 | Train Acc: 0.9885 | Val Acc: 0.9765
Epoch 07 | Train Acc: 0.9907 | Val Acc: 0.9790
Epoch 08 | Train Acc: 0.9916 | Val Acc: 0.9777
Epoch 09 | Train Acc: 0.9933 | Val Acc: 0.9770
Epoch 10 | Train Acc: 0.9922 | Val Acc: 0.9753

=== ReLU + Adam + L2 ===
Epoch 01 | Train Acc: 0.9268 | Val Acc: 0.9610
Epoch 02 | Train Acc: 0.9701 | Val Acc: 0.9638
Epoch 03 | Train Acc: 0.9784 | Val Acc: 0.9695
Epoch 04 | Train Acc: 0.9819 | Val Acc: 0.9733
Epoch 05 | Train Acc: 0.9853 | Val Acc: 0.9697
Epoch 06 | Train Acc: 0.9878 | Val Acc: 0.9763
Epoch 07 | Train Acc: 0.9886 | Val Acc: 0.9737
Epoch 08 | Train Acc: 0.9902 | Val Acc: 0.9708
Epoch 09 | Train Acc: 0.9910 | Val Acc: 0.9775
Epoch 10 | Train Acc: 0.9912 | Val Acc: 0.9745

=== ReLU + Adam + Dropout ===
Epoch 01 | Train Acc: 0.8966 | Val Acc: 0.9528
Epoch 02 | Train Acc: 0.9448 | Val Acc: 0.9633
Epoch 03 | Train Acc: 0.9537 | Val Acc: 0.9662
Epoch 04 | Train Acc: 0.9578 | Val Acc: 0.9678
Epoch 05 | Train Acc: 0.9629 | Val Acc: 0.9758
```

```
Epoch 06 | Train Acc: 0.9647 | Val Acc: 0.9740
Epoch 07 | Train Acc: 0.9667 | Val Acc: 0.9772
Epoch 08 | Train Acc: 0.9678 | Val Acc: 0.9775
Epoch 09 | Train Acc: 0.9696 | Val Acc: 0.9742
Epoch 10 | Train Acc: 0.9709 | Val Acc: 0.9778

=== FINAL LOSSES SUMMARY ===
Sigmoid + SGD
  Final Training Loss : 0.0997
  Final Validation Loss : 0.1163
----------------------------
ReLU + Adam (No Reg)
  Final Training Loss : 0.0231
  Final Validation Loss :
  0.0998
----------------------------
ReLU + Adam + L2
  Final Training Loss : 0.0272
  Final Validation Loss :
  0.0928
----------------------------
ReLU + Adam + Dropout
  Final Training Loss : 0.0906
  Final Validation Loss :
  0.0701
----------------------------

Plots saved in ./plots/
```

Results :

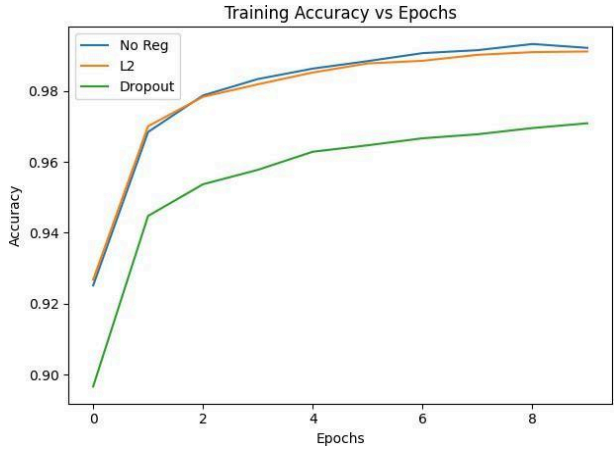
Sigmoid + SGD shows low training and validation accuracy with high loss. It converges slowly due to vanishing gradients and results in underfitting.

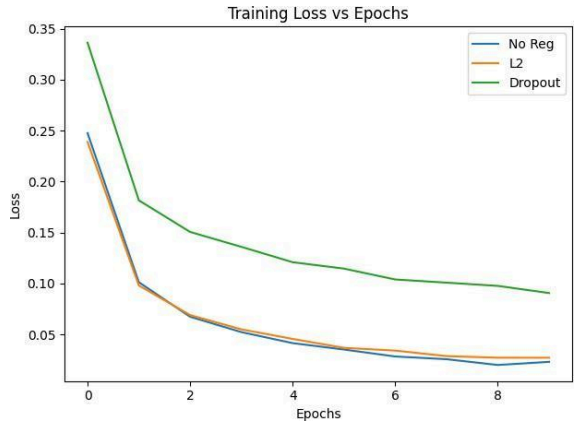
ReLU + Adam (No Regularization) achieves very high training accuracy and very low training loss with fast convergence. However, validation performance is lower, indicating overfitting.

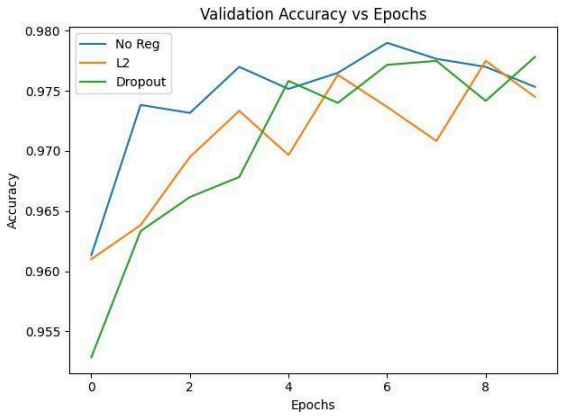
ReLU + Adam + L2 slightly increases training loss but improves validation accuracy. L2 regularization reduces overfitting by penalizing large weights.

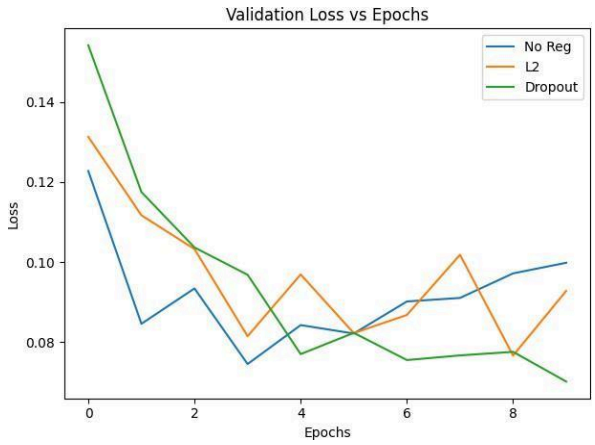
ReLU + Adam + Dropout has higher training loss but achieves the lowest validation loss and best validation accuracy, showing excellent generalization with minimal overfitting.

Among all models, **ReLU + Adam + Dropout** performs the best as it achieves the **lowest validation loss (0.0701)**, indicating better generalization and reduced overfitting compared to other approaches.


Training Accuracy vs Epochs
No Reg
L2
Dropout
Accuracy
Epochs


Training Loss vs Epochs
No Reg
L2
Dropout
Loss
Epochs


Validation Accuracy vs Epochs
No Reg
L2
Dropout
Accuracy
Epochs


Validation Loss vs Epochs
No Reg
L2
Dropout
Loss
Epochs

Experiment 2

Aim:

Implementation of a 1D Convolutional Neural Network for Sequence-Based Prediction Tasks

Theory:

A **1D Convolutional Neural Network (1D-CNN)** is a type of deep learning model designed to process **sequential data** such as time series, text, or signals. Unlike traditional neural networks, 1D-CNNs apply convolution operations along a single dimension, making them efficient for capturing patterns in sequences.

The model consists of the following layers:

- **Convolutional Layer**: Applies filters (kernels) that slide over the input sequence to extract important features such as local patterns.

Feature Map=Input∗Kernel

- **Activation Function**: Introduces non-linearity (commonly ReLU).
- **Pooling Layer**: Reduces dimensionality and helps retain important features (e.g., Max Pooling).
- **Fully Connected Layer**: Flattens the extracted features and produces the final output.

Examples:

- Time-series forecasting
- Text classification
- Signal processing

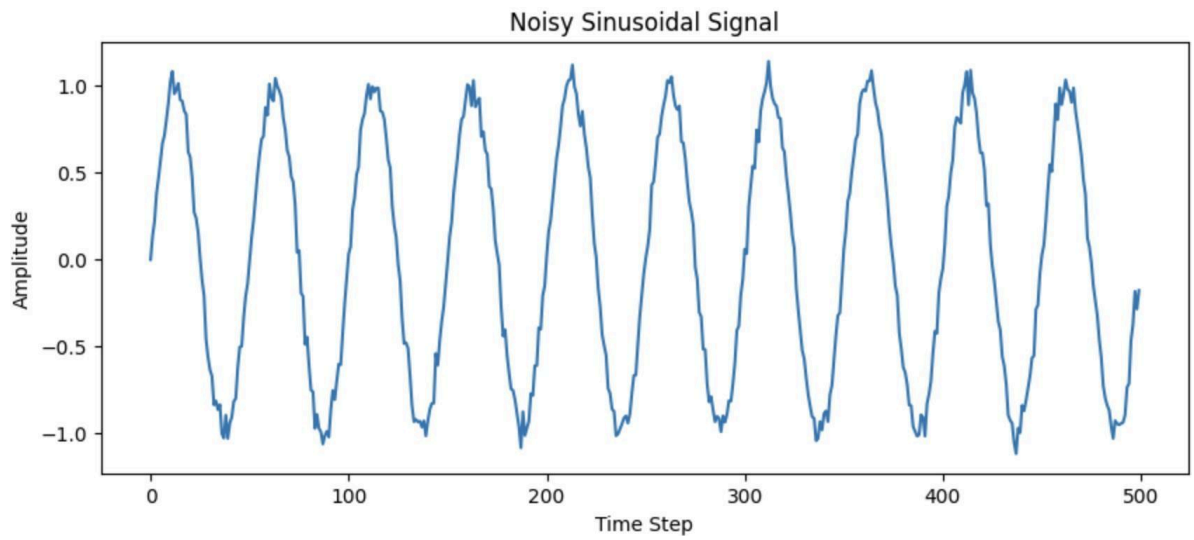
Code:

```
import torch
import torch.nn as nn
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split

# Parameters
num_samples = 5000
frequency = 2
sampling_rate = 100
noise_level = 0.05

# Time axis
t = np.linspace(0, num_samples / sampling_rate, num_samples)

# Clean sine wave
signal = np.sin(2 * np.pi * frequency * t)
```

```
# Add noise
signal += noise_level * np.random.randn(num_samples)

# Visualize
plt.figure(figsize=(10, 4))
plt.plot(signal[:500])
plt.title("Noisy Sinusoidal Signal")
plt.xlabel("Time Step")
plt.ylabel("Amplitude")
plt.show()
#window
time_steps = 50
X = []
y = []

for i in range(len(signal) - time_steps):
    X.append(signal[i:i + time_steps]) # past sequence
    y.append(signal[i + time_steps])     # next value

X = np.array(X)
y = np.array(y)

# Reshape for 1D CNN: (samples, time_steps, features)
X = X.reshape(X.shape[0], time_steps, 1)

print("X shape:", X.shape)
print("y shape:", y.shape)

#train test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
#model
model = nn.Sequential(
    nn.Conv1d(in_channels=1, out_channels=32, kernel_size=3),
    nn.ReLU(),
    nn.MaxPool1d(kernel_size=2),

    nn.Conv1d(in_channels=32, out_channels=64, kernel_size=3),
    nn.ReLU(),
    nn.MaxPool1d(kernel_size=2),

    nn.Flatten(),

    nn.Linear(64 * ((time_steps - 2) // 2 - 2) // 2, 128),
    nn.ReLU(),

    nn.Linear(128, 1) # Linear output for regression
)
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

# Convert TRAIN / TEST to torch
```

```
X_train_torch = torch.tensor(X_train, dtype=torch.float32).permute(0, 2, 1)
y_train_torch = torch.tensor(y_train, dtype=torch.float32).unsqueeze(1)
X_test_torch = torch.tensor(X_test, dtype=torch.float32).permute(0, 2, 1)
y_test_torch = torch.tensor(y_test, dtype=torch.float32).unsqueeze(1)
# Training Loop
epochs = 30
batch_size = 32

for epoch in range(epochs):
    model.train()
    perm = torch.randperm(X_train_torch.size(0))
    train_loss = 0.0

    for i in range(0, X_train_torch.size(0), batch_size):
        idx = perm[i:i + batch_size]
        xb = X_train_torch[idx]
        yb = y_train_torch[idx]

        optimizer.zero_grad()
        preds = model(xb)
        loss = criterion(preds, yb)
        loss.backward()
        optimizer.step()

        train_loss += loss.item()
    train_loss /= (X_train_torch.size(0) / batch_size)
    # Validation
    model.eval()
    with torch.no_grad():
        val_preds = model(X_test_torch)
        val_loss = criterion(val_preds, y_test_torch)

    print(
        f"Epoch [{epoch+1}/{epochs}] "
        f"Train MSE: {train_loss:.6f} "
        f"Val MSE: {val_loss.item():.6f}"
    )


# Evaluation & Visualization
model.eval()
with torch.no_grad():
    predictions = model(X_test_torch).squeeze().numpy()

# Plot true vs predicted values
plt.figure(figsize=(10, 4))
plt.plot(y_test[:200], label="True")
plt.plot(predictions[:200], label="Predicted")
plt.legend()
```

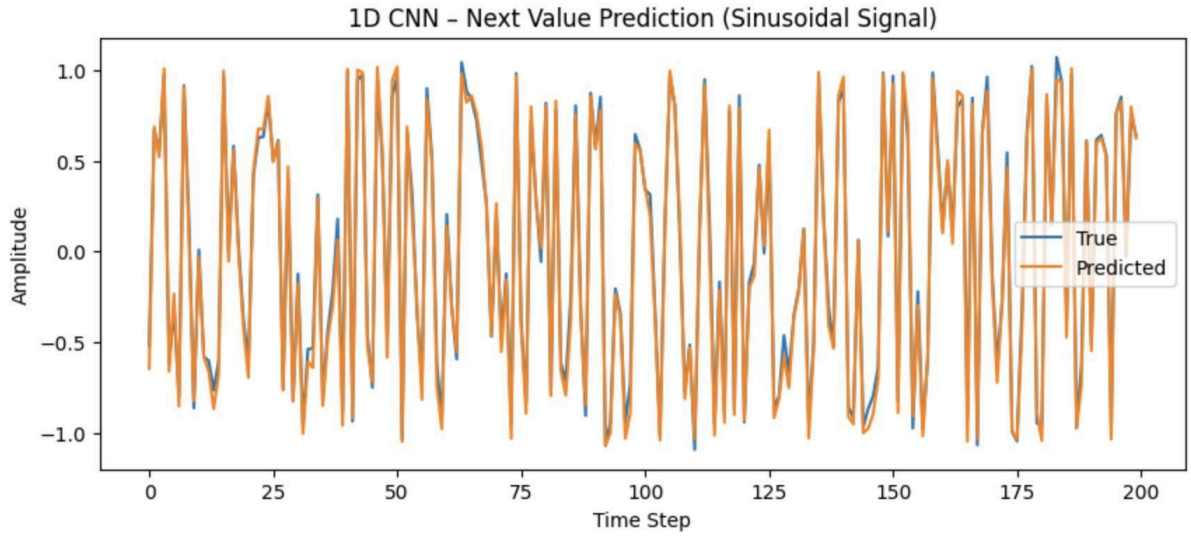
```
plt.title("1D CNN – Next Value Prediction (Sinusoidal Signal)")
plt.xlabel("Time Step")
plt.ylabel("Amplitude")
plt.show()
```




```
Epoch [1/30] Train MSE: 0.019346 Val MSE: 0.002680
Epoch [2/30] Train MSE: 0.003132 Val MSE: 0.003537
Epoch [3/30] Train MSE: 0.002954 Val MSE: 0.003192
Epoch [4/30] Train MSE: 0.003234 Val MSE: 0.002794
Epoch [5/30] Train MSE: 0.003015 Val MSE: 0.003402
Epoch [6/30] Train MSE: 0.003044 Val MSE: 0.002671
Epoch [7/30] Train MSE: 0.003099 Val MSE: 0.002847
Epoch [8/30] Train MSE: 0.003011 Val MSE: 0.003039
Epoch [9/30] Train MSE: 0.002962 Val MSE: 0.002758
Epoch [10/30] Train MSE: 0.003214 Val MSE: 0.003078
Epoch [11/30] Train MSE: 0.003294 Val MSE: 0.003295
Epoch [12/30] Train MSE: 0.003091 Val MSE: 0.003056
Epoch [13/30] Train MSE: 0.002978 Val MSE: 0.004878
Epoch [14/30] Train MSE: 0.003056 Val MSE: 0.002548
Epoch [15/30] Train MSE: 0.003127 Val MSE: 0.003275
Epoch [16/30] Train MSE: 0.003242 Val MSE: 0.002582
Epoch [17/30] Train MSE: 0.003002 Val MSE: 0.002601
Epoch [18/30] Train MSE: 0.002829 Val MSE: 0.002802
Epoch [19/30] Train MSE: 0.003054 Val MSE: 0.002826
Epoch [20/30] Train MSE: 0.003008 Val MSE: 0.002661
Epoch [21/30] Train MSE: 0.002928 Val MSE: 0.002935
Epoch [22/30] Train MSE: 0.003047 Val MSE: 0.002783
Epoch [23/30] Train MSE: 0.003100 Val MSE: 0.002666
Epoch [24/30] Train MSE: 0.002972 Val MSE: 0.002795
Epoch [25/30] Train MSE: 0.002948 Val MSE: 0.002591
Epoch [26/30] Train MSE: 0.002842 Val MSE: 0.002842
Epoch [27/30] Train MSE: 0.002801 Val MSE: 0.002891
Epoch [28/30] Train MSE: 0.002966 Val MSE: 0.003117
Epoch [29/30] Train MSE: 0.002966 Val MSE: 0.002639
Epoch [30/30] Train MSE: 0.002869 Val MSE: 0.003570
```

Output:

...



Result:

The model demonstrated rapid convergence, achieving a final Training MSE of **0.002869** and a final Validation MSE of **0.003570**. Throughout the training process, the Training and Validation loss curves tracked closely, suggesting that the model successfully learned the underlying signal pattern without overfitting to the added Gaussian noise.

The convergence behavior suggests that the chosen window size of 50 steps provided sufficient temporal context for the 1D-CNN to distinguish the signal trend from the high-frequency noise component. These results confirm the effectiveness of 1D convolutional layers in feature extraction for time-series regression tasks.

Experiment 3

Aim - Design and Implementation of a Deep CNN with Padding and Batch Normalization for Multi-Class Image Classification

Theory -

A **Convolutional Neural Network (CNN)** is a deep learning model widely used for image classification tasks. It automatically learns spatial features using convolution operations.

CNN Architecture Components

1. **Convolution Layer**
   Applies filters to extract features such as edges, textures, and patterns.
2. **Padding**
   Padding adds extra pixels (usually zeros) around the input image to:
   - Preserve spatial dimensions
   - Prevent information loss at borders
3. **Activation Function (ReLU)**
   Introduces non-linearity:
   f(x)=max(0,x)
4. **Pooling Layer**
   Reduces spatial dimensions and computation.
5. **Batch Normalization**
   Normalizes activations across a batch to:
   - Speed up training
   - Improve stability
   - Reduce internal covariate shift
6. **Fully Connected Layer**
   Flattens features and performs classification.
7. **Softmax Function**
   Converts outputs into probabilities for multi-class classification:

$$f(x_i) = \frac{e^{x_i}}{\sum e^{x_j}}$$

Working of CNN

1. Input image is passed through convolution layers
2. Padding preserves feature size
3. Activation (ReLU) is applied
4. Batch normalization stabilizes outputs
5. Pooling reduces dimensions
6. Fully connected layer classifies into multiple classes

Code and Output:

Imports

```
import torch

import torch.nn as nn

import torch.optim as

optim import torchvision

from torch.utils.data import DataLoader, TensorDataset, random_split

import matplotlib.pyplot as plt

import numpy as np

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
```

Load CIFAR-10

```
train_raw = torchvision.datasets.CIFAR10(

    root="./data", train=True, download=True

)
```

COMPUTE MEAN & STD

```
sum_rgb = torch.zeros(3)

sum_sq_rgb = torch.zeros(3)

num_pixels = 0
```

```
for img, _ in train_raw:
    img = torch.tensor(np.array(img), dtype=torch.float32) / 255.0
    sum_rgb += img.sum(dim=(0, 1))
    sum_sq_rgb += (img ** 2).sum(dim=(0, 1))
    num_pixels += img.shape[0] * img.shape[1]
mean = sum_rgb / num_pixels
std = torch.sqrt(sum_sq_rgb / num_pixels - mean ** 2)

print("Computed Mean :", mean)
print("Computed Std :", std)
```

```
Computed Mean : tensor([0.4914, 0.4822, 0.4465])
Computed Std : tensor([0.2470, 0.2435, 0.2616])
```

Preprocessing

```
def preprocess(dataset, mean, std):
    images, labels = [], []
    for img, label in dataset:
        img = torch.tensor(np.array(img), dtype=torch.float32) / 255.0
        img = img.permute(2, 0, 1)
        img = (img - mean[:, None, None]) / std[:, None, None]
        images.append(img)
        labels.append(label)
    return torch.stack(images), torch.tensor(labels)
X_train, y_train = preprocess(train_raw, mean, std)
dataset = TensorDataset(X_train, y_train)
train_ds, val_ds = random_split(dataset, [45000, 5000])
```

```
train_loader = DataLoader(train_ds, batch_size=128, shuffle=True)

val_loader = DataLoader(val_ds, batch_size=128)
```

CNN Model

```
class CNN(nn.Module):

    def __init__(self):

        super(CNN, self).__init__()

        self.conv_block = nn.Sequential(

            # Input -> 32 * 32 * 3

            # Block 1

            nn.Conv2d(3, 32, kernel_size=3, stride=1, padding=1),

            nn.BatchNorm2d(32),

            nn.ReLU(),

            # output -> 32 * 32 * 32

            # Block 2 (stride=2 reduces size)

            nn.Conv2d(32, 64, kernel_size=3, stride=2, padding=1),

            nn.BatchNorm2d(64),

            nn.ReLU(),

            # output -> 16 * 16 * 64

            # Block 3 (stride=2 reduces again)

            nn.Conv2d(64, 128, kernel_size=3, stride=2, padding=1),

            nn.BatchNorm2d(128),

            nn.ReLU()

             # Output 8 * 8 * 128

        )
```

```
        self.fc = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 8 * 8, 256),
            nn.ReLU(),
            nn.Linear(256, 10)
        )
    def forward(self, x):
        x = self.conv_block(x)
        x = self.fc(x)
        return x
model = CNN().to(device)
```

```
import torch
import torch.nn as nn
import torchvision.models as models
```

Loss And Optimiser

```
criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
```

Training

```
def run_epoch(model, loader, train=True):
    if train:
        model.train()
    else:
        model.eval()
```

```
    total_loss = 0.0
    correct = 0
    total = 0
    for x, y in loader:
        x, y = x.to(device), y.to(device)
        if train:
            optimizer.zero_grad()
            outputs = model(x)
            loss = criterion(outputs, y)
            loss.backward()
            optimizer.step()
        else:
            with torch.no_grad():
                outputs = model(x)
                loss = criterion(outputs, y)
        total_loss += loss.item() * y.size(0)
        correct += (outputs.argmax(1) == y).sum().item()
        total += y.size(0)
    return total_loss / total, correct / total
```

```
Epoch 01 | Train Loss: 1.2413 | Val Loss: 1.0391 | Train Acc: 0.5469 | Val Acc: 0.6300
Epoch 02 | Train Loss: 0.7452 | Val Loss: 0.7437 | Train Acc: 0.7356 | Val Acc: 0.7412
Epoch 03 | Train Loss: 0.5536 | Val Loss: 0.6482 | Train Acc: 0.8065 | Val Acc: 0.7798
Epoch 04 | Train Loss: 0.4229 | Val Loss: 0.6118 | Train Acc: 0.8523 | Val Acc: 0.7906
Epoch 05 | Train Loss: 0.3108 | Val Loss: 0.5544 | Train Acc: 0.8912 | Val Acc: 0.8190
Epoch 06 | Train Loss: 0.2196 | Val Loss: 0.5982 | Train Acc: 0.9212 | Val Acc: 0.8188
Epoch 07 | Train Loss: 0.1582 | Val Loss: 0.7003 | Train Acc: 0.9432 | Val Acc: 0.8060
Epoch 08 | Train Loss: 0.1122 | Val Loss: 0.7320 | Train Acc: 0.9604 | Val Acc: 0.8118
Epoch 09 | Train Loss: 0.0870 | Val Loss: 0.7256 | Train Acc: 0.9688 | Val Acc: 0.8212
```

Performance:

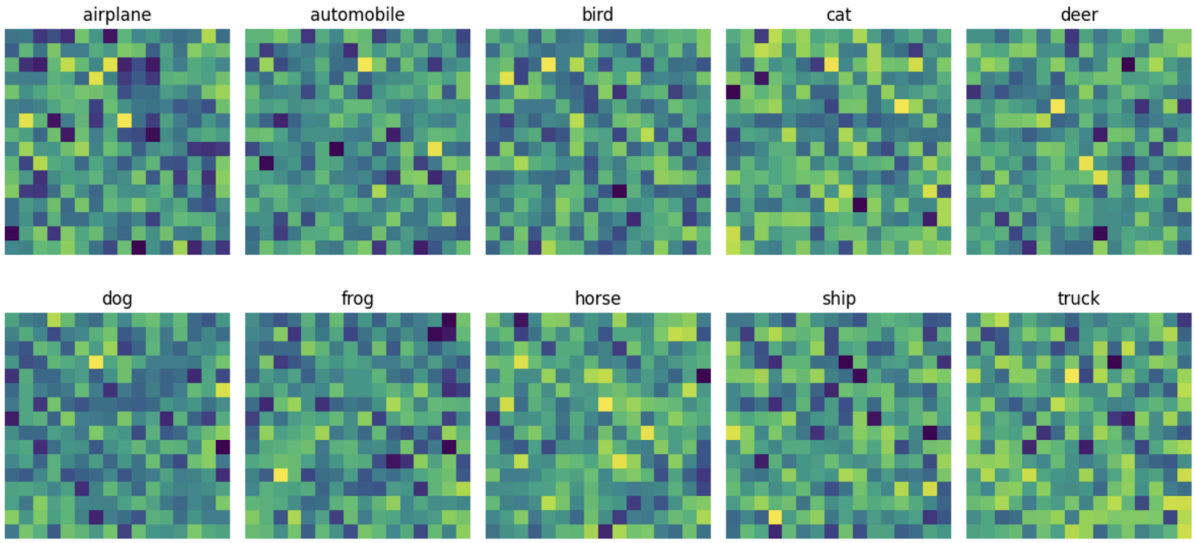
```
print(f"CNN Model     | Val Acc:
{best_val_acc:.4f}") CNN Model | Val Acc: 0.8028
```

Plots :

```
import matplotlib.pyplot as plt
model.eval()

# CIFAR-10 class names
class_names = [
  "airplane", "automobile", "bird", "cat", "deer",
  "dog", "frog", "horse", "ship", "truck"
]
fc_weights = model.fc[3].weight.data.cpu()
fc_weights = (fc_weights - fc_weights.min()) / (fc_weights.max() - fc_weights.min())
class_maps = fc_weights.view(10, 16, 16)
for i in range(10):
  plt.figure(figsize=(6,4))
  plt.imshow(class_maps[i])
  plt.axis("off")
  plt.show()
```

Weights visualized are as follows :

```
['airplane', 'automobile', 'bird', 'cat', 'deer', 'dog', 'frog', 'horse', 'ship', 'truck']
```



Results:

After training the CNN model for 9 epochs, the following performance was observed:

- **Final Training Loss**: 0.0870
- **Final Validation Loss**: 0.7256
- **Final Training Accuracy**: 96.88%
- **Final Validation Accuracy**: 82.12%

Training loss consistently decreased, indicating effective learning. Validation loss decreased initially but increased after epoch 5, suggesting **overfitting**. Training accuracy improved steadily and reached a high value (≈97%). Validation accuracy peaked around **82%**, showing good but limited generalization.

Experiment 4

Aim:

Design and Implementation of a CNN Model for Text Classification

Theory:

A **Convolutional Neural Network (CNN)** can be effectively used for **text classification**, where the goal is to assign predefined categories (e.g., spam/ham, sentiment) to textual data. Instead of images, CNNs process text by treating it as a sequence of words or tokens.

Working of CNN for Text

1. **Text Representation (Embedding Layer):**
   Words are converted into dense vectors using an embedding layer, forming a matrix representation of the text.

2. **Convolutional Layer:**
   1D filters slide over embeddings to capture local features such as phrases or n-grams.

3. **Activation Function:**
   ReLU introduces non-linearity to learn complex patterns.

4. **Pooling Layer:**
   Adaptive max pooling is used to reduce each feature map to a single important value.

5. **Fully Connected Layer:**
   The output is squeezed and passed to a dense layer for classification.

6. **Output Layer:**
   Sigmoid activation is used to produce a probability for binary classification.

Code:

```
!pip install datasets

import torch
import torch.nn as nn
import torch.optim as
optim
from datasets import load_dataset
from torch.utils.data import DataLoader
from torch.nn.utils.rnn import pad_sequence
from collections import Counter

# ---------------
# 1. Load IMDb Dataset
# ---------------
dataset = load_dataset("imdb")
```

```
train_data = dataset["train"]
test_data = dataset["test"]

# ----------------
# 2. Simple Tokenizer
# ----------------
def tokenize(text):
    return text.lower().split()

# ----------------
# 3. Build Vocabulary (Top 20000 words)
# ----------------
counter = Counter()

for item in train_data:
    counter.update(tokenize(item["text"]))

vocab = {"<pad>": 0, "<unk>": 1}

for word, freq in counter.most_common(20000):
    vocab[word] = len(vocab)
vocab_size = len(vocab)
#
# 4. Encode Function
# ----------------
def encode(text):
    return [vocab.get(word, 1) for word in tokenize(text)]

# ----------------
# 5. Collate Function
# ----------------
def collate_batch(batch):
    texts = []
    labels = []

    for item in batch:
        texts.append(torch.tensor(encode(item["text"]), dtype=torch.long))
        labels.append(torch.tensor(item["label"], dtype=torch.float))

    texts = pad_sequence(texts, batch_first=True)
    labels = torch.tensor(labels)

    return texts, labels

# ----------------
# 6. DataLoaders
# ----------------
train_loader = DataLoader(train_data, batch_size=64,
                          shuffle=True, collate_fn=collate_batch)
```

```
test_loader = DataLoader(test_data, batch_size=64,
                         shuffle=False, collate_fn=collate_batch)

print("Setup Complete ”)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# - - - - - - - - - - - - - - -
# CNN Model (No class)
# - - - - - - - - - - - - - - -
embedding_dim = 128

embedding = nn.Embedding(vocab_size, embedding_dim, padding_idx=0)
conv = nn.Conv1d(embedding_dim, 128, kernel_size=5)
relu = nn.ReLU()
pool = nn.AdaptiveMaxPool1d(1)
fc = nn.Linear(128, 1)
sigmoid = nn.Sigmoid()

embedding.to(device)
conv.to(device)
relu.to(device)
pool.to(device)
fc.to(device)
sigmoid.to(device)

criterion = nn.BCELoss()
optimizer = optim.Adam(
    list(embedding.parameters()) +
    list(conv.parameters()) +
    list(fc.parameters()),
    lr=0.001
)

# - - - - - - - - - - - - - - -
# Training
# - - - - - - - - - - - - - - -
for epoch in range(10):
    total_loss = 0

    for texts, labels in train_loader:
        texts, labels = texts.to(device), labels.to(device)

        # Forward
        x = embedding(texts)          # (batch, seq_len,
        embed_dim) x = x.permute(0, 2, 1)     # (batch,
        embed_dim, seq_len) x = conv(x)
        x = relu(x)
        x =
        pool(x)
        x = x.squeeze(2)
        x = fc(x)
        outputs = sigmoid(x).squeeze()

        loss = criterion(outputs, labels)
```

```
        # Backward
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        total_loss += loss.item()

    print(f"Epoch {epoch+1}, Loss: {total_loss/len(train_loader):.4f}")

# ---------------
# Evaluation
# ---------------
correct = 0
total = 0

with torch.no_grad():
    for texts, labels in test_loader:
        texts, labels = texts.to(device), labels.to(device)

        x  = embedding(texts)
        x = x.permute(0, 2, 1)
        x = conv(x)
        x = relu(x)
        x =
        pool(x)
        x = x.squeeze(2)
        x = fc(x)
        outputs = sigmoid(x).squeeze()

        predictions = (outputs > 0.5).float()

        correct += (predictions == labels).sum().item()
        total += labels.size(0)

print("Test Accuracy:", correct / total)
```

Output:

```
README.md: 7.81k/? [00:00<00:00, 653kB/s]
plain_text/train-00000-of-00001.parquet: 100% 21.0M/21.0M [00:00<00:00, 24.6MB/s]
plain_text/test-00000-of-00001.parquet: 100% 20.5M/20.5M [00:00<00:00, 186MB/s]
Notebook editor cells
plain_text/unsupervised-00000-of-00001.p(...): 100% 42.0M/42.0M [00:00<00:00, 160MB/s]
Generating train split: 100% 25000/25000 [00:00<00:00, 124536.12 examples/s]
Generating test split: 100% 25000/25000 [00:00<00:00, 179817.79 examples/s]
Generating unsupervised split: 100% 50000/50000 [00:00<00:00, 180195.37 examples/s]
Setup Complete ✅
```

```
Epoch 1, Loss: 0.5207
Epoch 2, Loss: 0.3331
Epoch 3, Loss: 0.1971
Epoch 4, Loss: 0.0944
Epoch 5, Loss: 0.0376
Epoch 6, Loss: 0.0154
Epoch 7, Loss: 0.0077
Epoch 8, Loss: 0.0048
Epoch 9, Loss: 0.0031
Epoch 10, Loss: 0.0022
Test Accuracy: 0.8662
```

Result:

The model was trained for **10 epochs**, and the training loss showed a consistent decrease, indicating effective learning.

- **Final Training Loss:** 0.0022
- **Test Accuracy:** 86.62%

The results demonstrate that the model is able to effectively classify movie reviews into positive and negative sentiments with good accuracy.

Experiment - 5

Aim - Implementation of Standard VGG-16 and VGG-19 Architectures for Multi-Class Image Classification

Theory -

Key Features of VGG Networks :

- Uses 3×3 convolution filters throughout
- Applies ReLU activation after each convolution
- Uses Max Pooling (2×2) to reduce dimensions
- Deep architecture with multiple layers
- Ends with Fully Connected layers + Softmax for classification

VGG-16 Architecture

VGG-16 consists of **16 weight layers**:

- 13 Convolutional layers
- 3 Fully Connected layers

Structure:

- Input → Conv → Conv → Pool
- Conv → Conv → Pool
- Conv → Conv → Conv → Pool
- Conv → Conv → Conv → Pool
- Conv → Conv → Conv → Pool
- Fully Connected → Softmax

3.4 VGG-19 Architecture

VGG-19 is a deeper version with **19 layers**:

- 16 Convolutional layers
- 3 Fully Connected layers

It follows the same structure as VGG-16 but with more convolution layers, allowing it to learn more complex features.

Code and Output :

Import Libraries :

```
import torch

import torch.nn as nn

import torch.optim as

optim

from torchvision import datasets, transforms

from torch.utils.data import DataLoader

from torch.cuda.amp import autocast,

GradScaler import torch.backends.cudnn as

cudnn batch_size = 128

num_epochs = 10

learning_rate = 0.001

num_classes = 10

image_size = 224

num_workers = 2

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

cudnn.benchmark = True

train_transform = transforms.Compose([

  transforms.RandomHorizontalFlip(),

  transforms.RandomCrop(32, padding=4),

  transforms.ToTensor(),

  transforms.Normalize(

     [0.4914, 0.4822, 0.4465],
```

```
        [0.2470, 0.2435, 0.2616]
    )
])
val_transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize(
        [0.4914, 0.4822, 0.4465],
        [0.2470, 0.2435, 0.2616]
    )
])
```

Dataset Loader :

```
train_dataset = datasets.CIFAR10(
    root="./data",
    train=True,
    transform=train_transform,
    download=True
)
val_dataset = datasets.CIFAR10(
    root="./data",
    train=False,
    transform=val_transform,
    download=True
```

```
)


train_loader = DataLoader(
    train_dataset,
    batch_size=batch_size,
    shuffle=True,
    num_workers=num_workers,
    pin_memory=True
)
val_loader = DataLoader(
    val_dataset,
    batch_size=batch_size,
    shuffle=False,
    num_workers=num_workers,
    pin_memory=True
)
```

VGG16 Model :

```
class VGG16(nn.Module):
    def __init__(self, num_classes=10):
        super(VGG16, self).__init__()
        self.features = self._make_layers([2,2,3,3,3])
        self.classifier = self._make_classifier(num_classes)
        self._initialize_weights()
```

```
def _make_layers(self, cfg):
    channels = [64,128,256,512,512]
    layers = []
    in_c = 3
    for num_convs, out_c in zip(cfg, channels):
        for _ in range(num_convs):
            layers.append(nn.Conv2d(in_c, out_c, 3, padding=1))
            layers.append(nn.ReLU(True))
            in_c = out_c
        layers.append(nn.MaxPool2d(2,2))
    return nn.Sequential(*layers)

def _make_classifier(self, num_classes):
    return nn.Sequential(
        nn.Linear(512, 512),
        nn.ReLU(True),
        nn.Dropout(0.5),
        nn.Linear(512, num_classes)
    )

def forward(self, x):
    x = self.features(x)
```

```
        x = torch.flatten(x, 1)
        return self.classifier(x)

    def _initialize_weights(self):
        for m in self.modules():
            if isinstance(m, nn.Conv2d):
                nn.init.kaiming_normal_(m.weight, nonlinearity='relu')
                nn.init.constant_(m.bias, 0)
            elif isinstance(m, nn.Linear):
                nn.init.normal_(m.weight, 0, 0.01)
                nn.init.constant_(m.bias, 0)
```

VGG19 Model :

```
class VGG19(nn.Module):
    def __init__(self, num_classes=10):
        super(VGG19, self).__init__()
        self.features = self._make_layers([2,2,4,4,4])
        self.classifier = self._make_classifier(num_classes)
        self._initialize_weights()

    def _make_layers(self, cfg):
        channels = [64,128,256,512,512]
        layers = []
```

```
    in_c = 3
    for num_convs, out_c in zip(cfg, channels):
        for _ in range(num_convs):
            layers.append(nn.Conv2d(in_c, out_c, 3, padding=1))
            layers.append(nn.ReLU(True))
            in_c = out_c
        layers.append(nn.MaxPool2d(2,2))
    return nn.Sequential(*layers)
def _make_classifier(self, num_classes):
    return nn.Sequential(
        nn.Linear(512, 512),
        nn.ReLU(True),
        nn.Dropout(0.5),
        nn.Linear(512, num_classes)
    )
def forward(self, x):
    x = self.features(x)
    x = torch.flatten(x, 1)
    return self.classifier(x)


def _initialize_weights(self):
    for m in self.modules():
        if isinstance(m, nn.Conv2d):
```

```
            nn.init.kaiming_normal_(m.weight, nonlinearity='relu')
            nn.init.constant_(m.bias, 0)
        elif isinstance(m, nn.Linear):
            nn.init.normal_(m.weight, 0, 0.01)
            nn.init.constant_(m.bias, 0)
def train_model(model, model_name, num_epochs=25):
    model = model.to(device)
    criterion = nn.CrossEntropyLoss()
    optimizer = optim.SGD(
        model.parameters(),
        lr=learning_rate,
        momentum=0.9,
        weight_decay=5e-4
    )
    scheduler = optim.lr_scheduler.StepLR(
        optimizer,
        step_size=5,
        gamma=0.1
    )
```

Training :

```
    print(f"\n===== Training {model_name} =====\n")
    best_val_loss = float('inf')
    best_model_path = f"{model_name}_best.pth"
```

```
for epoch in range(num_epochs):
    model.train()
    train_loss = 0.0
    correct = 0
    total = 0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        train_loss += loss.item()
        _, predicted = outputs.max(1)
        total += labels.size(0)
        correct += predicted.eq(labels).sum().item()
    train_acc = 100. * correct / total
    avg_train_loss = train_loss / len(train_loader)
```

Validation :

```
    model.eval()
    val_loss = 0.0
    correct = 0
    total = 0
```

```
with torch.no_grad():
    for images, labels in val_loader:
        images, labels = images.to(device), labels.to(device)
        outputs = model(images)
        loss = criterion(outputs, labels)
        val_loss += loss.item()
        _, predicted = outputs.max(1)
        total += labels.size(0)
        correct += predicted.eq(labels).sum().item()
val_acc = 100. * correct / total
avg_val_loss = val_loss / len(val_loader)
scheduler.step()


print(
    f"Epoch [{epoch+1}/{num_epochs}] "
    f"Train Loss: {avg_train_loss:.4f} "
    f"Train Acc: {train_acc:.2f}% | "
    f"Val Loss: {avg_val_loss:.4f} "
    f"Val Acc: {val_acc:.2f}%"
)
# Save best model
if avg_val_loss < best_val_loss:
    best_val_loss = avg_val_loss
```

```
        torch.save(model.state_dict(), best_model_path)

    # Load best model at end

    model.load_state_dict(torch.load(best_model_path))

    print(f"\nBest model loaded from {best_model_path}")

vgg16 = VGG16(num_classes=10)

train_model(vgg16, "VGG16_CIFAR10", patience=3)
```

```
Epoch [1/50] Train Loss: 1.9811 Train Acc: 23.37% | Val Loss: 1.6387 Val Acc: 37.44%

Epoch [2/50] Train Loss: 1.5752 Train Acc: 40.96% | Val Loss: 1.5043 Val Acc: 45.53%

Epoch [3/50] Train Loss: 1.3240 Train Acc: 51.41% | Val Loss: 1.1861 Val Acc: 57.56%

Epoch [4/50] Train Loss: 1.1244 Train Acc: 59.71% | Val Loss: 0.9479 Val Acc: 66.45%

Epoch [5/50] Train Loss: 0.9691 Train Acc: 65.64% | Val Loss: 0.8850 Val Acc: 68.41%

Epoch [6/50] Train Loss: 0.7699 Train Acc: 72.75% | Val Loss: 0.7458 Val Acc: 73.50%

Epoch [7/50] Train Loss: 0.7266 Train Acc: 74.46% | Val Loss: 0.7289 Val Acc: 74.47%

Epoch [8/50] Train Loss: 0.6979 Train Acc: 75.40% | Val Loss: 0.7067 Val Acc: 75.54%

Epoch [9/50] Train Loss: 0.6777 Train Acc: 76.20% | Val Loss: 0.6763 Val Acc: 76.47%

Epoch [10/50] Train Loss: 0.6602 Train Acc: 76.84% | Val Loss: 0.6810 Val Acc: 76.29%

Epoch [11/50] Train Loss: 0.6275 Train Acc: 78.05% | Val Loss: 0.6504 Val Acc: 77.45%

Epoch [12/50] Train Loss: 0.6229 Train Acc: 78.34% | Val Loss: 0.6494 Val Acc: 77.33%

Epoch [13/50] Train Loss: 0.6172 Train Acc: 78.20% | Val Loss: 0.6481 Val Acc: 77.39%

Epoch [14/50] Train Loss: 0.6171 Train Acc: 78.34% | Val Loss: 0.6419 Val Acc: 77.83%

Epoch [15/50] Train Loss: 0.6110 Train Acc: 78.53% | Val Loss: 0.6474 Val Acc: 77.43%

Epoch [16/50] Train Loss: 0.6059 Train Acc: 78.68% | Val Loss: 0.6429 Val Acc: 77.66%

Epoch [17/50] Train Loss: 0.6074 Train Acc: 78.69% | Val Loss: 0.6421 Val Acc: 77.69%
```

🛑 Early stopping triggered.

```
vgg19 = VGG19(num_classes=10)

train_model(vgg19, "VGG19_CIFAR10", patience=3)
```

===== Training VGG19_CIFAR10 =====

Epoch [1/50] Train Loss: 2.0590 Train Acc: 20.44% | Val Loss: 1.7121 Val Acc: 34.24%

Epoch [2/50] Train Loss: 1.6489 Train Acc: 36.59% | Val Loss: 1.5160 Val Acc: 43.71%

Epoch [3/50] Train Loss: 1.4156 Train Acc: 47.43% | Val Loss: 1.2688 Val Acc: 52.88%

Epoch [4/50] Train Loss: 1.1989 Train Acc: 56.71% | Val Loss: 1.0670 Val Acc: 61.93%

Epoch [5/50] Train Loss: 1.0415 Train Acc: 62.87% | Val Loss: 0.9863 Val Acc: 65.16%

Epoch [6/50] Train Loss: 0.8204 Train Acc: 71.14% | Val Loss: 0.7815 Val Acc: 72.76%

Epoch [7/50] Train Loss: 0.7681 Train Acc: 72.95% | Val Loss: 0.7637 Val Acc: 73.30%

Epoch [8/50] Train Loss: 0.7439 Train Acc: 73.79% | Val Loss: 0.7303 Val Acc: 74.65%

Epoch [9/50] Train Loss: 0.7234 Train Acc: 74.63% | Val Loss: 0.7367 Val Acc: 74.31%

Epoch [10/50] Train Loss: 0.6948 Train Acc: 75.69% | Val Loss: 0.7130 Val Acc: 75.58%

Epoch [11/50] Train Loss: 0.6650 Train Acc: 76.88% | Val Loss: 0.6806 Val Acc: 76.54%

Epoch [12/50] Train Loss: 0.6592 Train Acc: 76.92% | Val Loss: 0.6777 Val Acc: 76.62%

Epoch [13/50] Train Loss: 0.6562 Train Acc: 77.11% | Val Loss: 0.6791 Val Acc: 76.50%

Epoch [14/50] Train Loss: 0.6513 Train Acc: 77.33% | Val Loss: 0.6718 Val Acc: 76.99%

Epoch [15/50] Train Loss: 0.6481 Train Acc: 77.18% | Val Loss: 0.6724 Val Acc: 76.93%

Epoch [16/50] Train Loss: 0.6423 Train Acc: 77.54% | Val Loss: 0.6704 Val Acc: 77.13%

Epoch [17/50] Train Loss: 0.6423 Train Acc: 77.36% | Val Loss: 0.6704 Val Acc: 77.08%

Epoch [18/50] Train Loss: 0.6420 Train Acc: 77.41% | Val Loss: 0.6705 Val Acc: 77.01%

Epoch [19/50] Train Loss: 0.6454 Train Acc: 77.38% | Val Loss: 0.6702 Val Acc: 77.04%

Epoch [20/50] Train Loss: 0.6436 Train Acc: 77.50% | Val Loss: 0.6692 Val Acc: 77.06%

Epoch [21/50] Train Loss: 0.6437 Train Acc: 77.53% | Val Loss: 0.6693 Val Acc: 77.04%

Epoch [22/50] Train Loss: 0.6440 Train Acc: 77.57% | Val Loss: 0.6694 Val Acc: 77.06%

Epoch [23/50] Train Loss: 0.6447 Train Acc: 77.42% | Val Loss: 0.6694 Val Acc: 77.03%

🛑 Early stopping triggered.

Results :

VGG-16 Performance

- Best Validation Loss: 0.6419 (Epoch 14)
- Final Training Loss: 0.6074
- Final Training Accuracy: 78.69%
- Best Validation Accuracy: 77.83%
- Observation: Achieved stable performance with relatively faster convergence.

VGG-19 Performance

- Best Validation Loss: 0.6692 (Epoch 20)
- Final Training Loss: 0.6447
- Final Training Accuracy: 77.42%
- Best Validation Accuracy: 77.13%
- Observation: Slightly slower convergence with marginally lower performance.

Comparison

Metric	VGG-16	VGG-19
Best Val Loss	0.6419	0.6692
Best Val Accuracy	77.83%	77.13%
Convergence Speed	Faster	Slower
Complexity	Lower	Higher

Experiment - 6

Aim - Implementation of a Recurrent Neural Network (RNN) for Sequential Data Analysis

Theory -

A **Recurrent Neural Network (RNN)** is a class of artificial neural networks designed for processing **sequential data** such as time series, text, or speech.

Unlike traditional feedforward neural networks, RNNs have **loops** that allow information to persist. This enables the network to maintain a **hidden state**, which acts as memory of previous inputs.

RNNs are widely used in applications like:

- Natural Language Processing (NLP)
- Time-series prediction
- Speech recognition
- Sequence classification

An RNN processes input sequences step-by-step. At each time step:

- Input: xtx_txt
- Hidden state: hth_tht
- Output: yty_tyt

The hidden state and Output is computed as:

$$h_t = f(W_h h_{t-1} + W_x x_t + b)$$

$$y_t = g(W_y h_t)$$

Where:

- Wh,Wx,Wy are weight matrices
- b is bias
- f, g are activation functions (like tanh, softmax)

Code:

```
import torch

import pandas as pd

import torch.nn as nn

import torch.optim as optim

import numpy as np

import yfinance as yf

from torch.utils.data import DataLoader, TensorDataset

from sklearn.preprocessing import MinMaxScaler

import matplotlib.pyplot as plt

# --- Configuration ---

WINDOW_SIZE = 10

BATCH_SIZE = 16

EPOCHS = 30

DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

# --- Step 1: Load Data ---

print("Downloading TSLA data...")

df = yf.download('TSLA', period='5y', interval='1d')

data = df['Close'].values.reshape(-1, 1)

# --- Step 2: Normalize ---

scaler = MinMaxScaler()

data_scaled = scaler.fit_transform(data)
```

```python
# --- Step 3: Split Data ---

train_size = int(len(data_scaled) * 0.8)

train_data = data_scaled[:train_size]

test_data = data_scaled[train_size:]

train_valid_size = int(len(train_data) * 0.8)

train_data_final = train_data[:train_valid_size]

valid_data = train_data[train_valid_size:]

# --- Step 4: Create Sequences ---

def create_sequences(data, window_size):

  X, y = [], []

  for i in range(len(data) - window_size):

    X.append(data[i:i + window_size])

    y.append(data[i + window_size])

  return torch.tensor(np.array(X), dtype=torch.float32), torch.tensor(np.array(y), dtype=torch.float32)

X_train, y_train = create_sequences(train_data_final, WINDOW_SIZE)

X_valid, y_valid = create_sequences(valid_data, WINDOW_SIZE)

X_test, y_test = create_sequences(test_data, WINDOW_SIZE)

train_loader = DataLoader(TensorDataset(X_train, y_train), batch_size=BATCH_SIZE, shuffle=True)

valid_loader = DataLoader(TensorDataset(X_valid, y_valid), batch_size=BATCH_SIZE)

test_loader = DataLoader(TensorDataset(X_test, y_test), batch_size=BATCH_SIZE)

# --- Step 5: Vanilla RNN Model ---

class RNNModel(nn.Module):

  def _init_(self, input_size=1, hidden_size=64, output_size=1):
```

```
        super(RNNModel, self).__init__()
        self.rnn = nn.RNN(input_size, hidden_size, batch_first=True)
        self.fc = nn.Linear(hidden_size, output_size)
    def forward(self, x):
        out, _ = self.rnn(x)
        out = out[:, -1, :] # Last timestep
        return self.fc(out)
model = RNNModel().to(DEVICE)
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)


# --- Step 6: Training ---
for epoch in range(EPOCHS):
    # TRAIN
    model.train()
    train_loss = 0
    for inputs, targets in train_loader:
        inputs, targets = inputs.to(DEVICE), targets.to(DEVICE)
        optimizer.zero_grad()
        outputs = model(inputs)
        loss = criterion(outputs, targets)
        loss.backward()
        # Gradient clipping (important for RNN)
```

```
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        optimizer.step()
        train_loss += loss.item()
    train_loss /= len(train_loader)
    # VALIDATION
    model.eval()
    val_loss = 0
    with torch.no_grad():
        for inputs, targets in valid_loader:
            inputs, targets = inputs.to(DEVICE), targets.to(DEVICE)
            outputs = model(inputs)
            loss = criterion(outputs, targets)
            val_loss += loss.item()
    val_loss /= len(valid_loader)
    print(f"Epoch {epoch+1}/{EPOCHS} | Train Loss: {train_loss:.6f} | Val Loss: {val_loss:.6f}")
# --- Step 7: Test Evaluation ---
model.eval()
test_loss = 0
mae = 0
with torch.no_grad():
    for inputs, targets in test_loader:
        inputs, targets = inputs.to(DEVICE), targets.to(DEVICE)
        outputs = model(inputs)
```

```
        test_loss += criterion(outputs, targets).item()

        mae += torch.mean(torch.abs(outputs - targets)).item()

test_loss /= len(test_loader)

mae /= len(test_loader)

rmse = np.sqrt(test_loss)

print("\n--- Test Metrics ---")

print(f"Test MSE : {test_loss:.6f}")

print(f"Test RMSE : {rmse:.6f}")

print(f"Test MAE : {mae:.6f}")

# --- Step 8: Visualization ---

with torch.no_grad():

    preds = model(X_test.to(DEVICE)).cpu().numpy()

    actuals = y_test.numpy()

# Denormalize

preds_rescaled = scaler.inverse_transform(preds)

actuals_rescaled =

scaler.inverse_transform(actuals)

plt.figure(figsize=(10, 5))

plt.plot(actuals_rescaled, label='Actual Price')

plt.plot(preds_rescaled, linestyle='--', label='Predicted Price')

plt.xlabel('Time (Days)')

plt.ylabel('Price (USD)')

plt.title('TSLA Price Prediction (Vanilla RNN)')

plt.legend()
```

```
plt.show()
```

Output :

```
Epoch 1/30 | Train Loss: 0.041630 | Val Loss: 0.030514

Epoch 2/30 | Train Loss: 0.005757 | Val Loss: 0.002807

Epoch 3/30 | Train Loss: 0.001379 | Val Loss: 0.002716

Epoch 4/30 | Train Loss: 0.001178 | Val Loss: 0.002678

Epoch 5/30 | Train Loss: 0.001358 | Val Loss: 0.002405

Epoch 6/30 | Train Loss: 0.001149 | Val Loss: 0.002628

Epoch 7/30 | Train Loss: 0.001247 | Val Loss: 0.002265

Epoch 8/30 | Train Loss: 0.001166 | Val Loss: 0.002303

Epoch 9/30 | Train Loss: 0.000930 | Val Loss: 0.002193

Epoch 10/30 | Train Loss: 0.000967 | Val Loss: 0.003430

Epoch 11/30 | Train Loss: 0.001024 | Val Loss: 0.002101

Epoch 12/30 | Train Loss: 0.000947 | Val Loss: 0.002420

Epoch 13/30 | Train Loss: 0.000832 | Val Loss: 0.001949

Epoch 14/30 | Train Loss: 0.000843 | Val Loss: 0.002222

Epoch 15/30 | Train Loss: 0.000947 | Val Loss: 0.001939

Epoch 16/30 | Train Loss: 0.000892 | Val Loss: 0.001948

Epoch 17/30 | Train Loss: 0.000903 | Val Loss: 0.001930

Epoch 18/30 | Train Loss: 0.000920 | Val Loss: 0.001781

Epoch 19/30 | Train Loss: 0.000742 | Val Loss: 0.001787

Epoch 20/30 | Train Loss: 0.000786 | Val Loss: 0.001774
```

Epoch 21/30 | Train Loss: 0.000945 | Val Loss: 0.001872

Epoch 22/30 | Train Loss: 0.000781 | Val Loss: 0.001781

Epoch 23/30 | Train Loss: 0.000727 | Val Loss: 0.001656

Epoch 24/30 | Train Loss: 0.000751 | Val Loss: 0.001751

Epoch 25/30 | Train Loss: 0.000740 | Val Loss: 0.001713

Epoch 26/30 | Train Loss: 0.000686 | Val Loss: 0.001700

Epoch 27/30 | Train Loss: 0.000692 | Val Loss: 0.001571

Epoch 28/30 | Train Loss: 0.000723 | Val Loss: 0.001538

Epoch 29/30 | Train Loss: 0.000685 | Val Loss: 0.001540

Epoch 30/30 | Train Loss: 0.000659 | Val Loss: 0.001505

--- Test Metrics ---

Test MSE : 0.001240

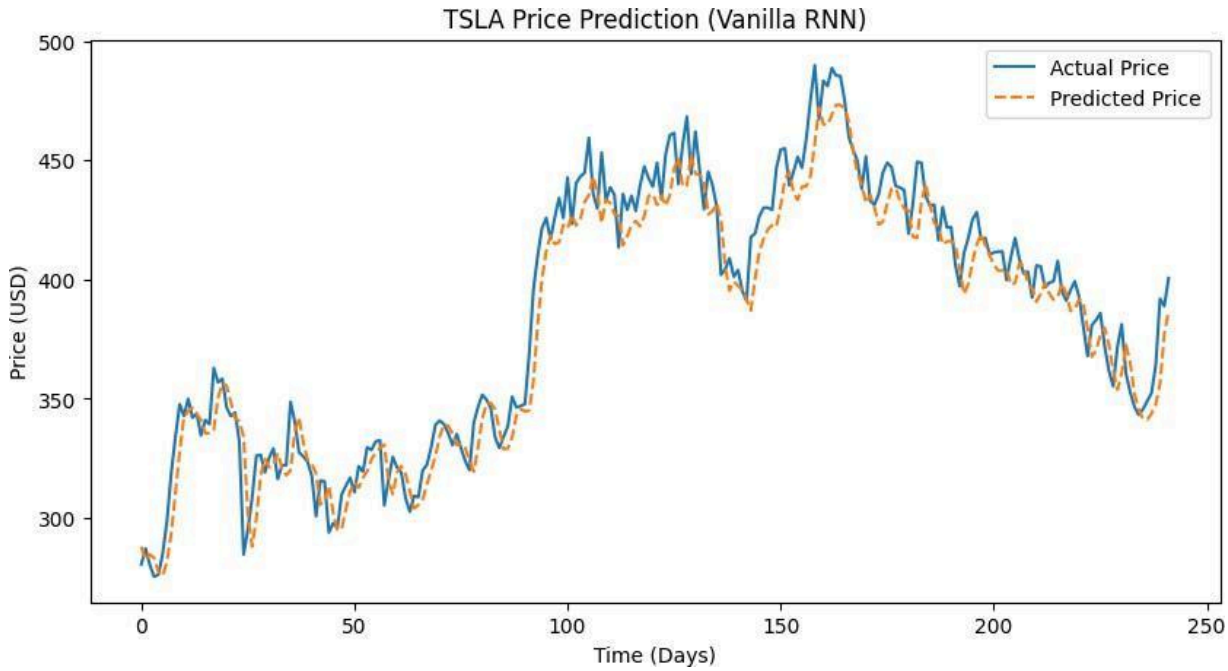
Test RMSE :

0.035209 Test MAE :

0.028067

Results :

The Vanilla RNN model was trained on 5 years of historical closing price data of Tesla Inc. and showed consistent convergence, with training loss decreasing from 0.0416 to 0.00066 and validation loss to 0.00150. On the test dataset, the model achieved an MSE of 0.00124, RMSE of 0.0352, and MAE of 0.0281 (in normalized scale), corresponding to an approximate prediction error of $8–10 in actual prices. The model successfully captured short-term trends but had limitations in handling sharp price fluctuations and long-term dependencies.


TSLA Price Prediction (Vanilla RNN)
Actual Price
Predicted Price
Price (USD)
Time (Days)
500
450
400
350
300
0
50
100
150
200
250

Experiment - 7

Aim - Sentiment Analysis Using a Recurrent Neural Network (RNN) on Movie Review Data

Theory -

Sentiment Analysis is a Natural Language Processing (NLP) technique used to determine whether a piece of text expresses a **positive or negative sentiment**. In this experiment, a **Recurrent Neural Network (RNN)** is used to classify movie reviews based on sentiment.

RNNs are suitable for text data because they process input sequentially and maintain a hidden state that captures information from previous words.

$$h_t = f(W_h h_{t-1} + W_x x_t + b)$$

Here, the hidden state ht helps the model understand context in a sentence.

The model typically consists of:

- **Embedding Layer** – converts words into vectors
- **RNN Layer** – captures sequence information
- **Dense Layer** – outputs sentiment prediction

The model is trained using **Binary Cross-Entropy Loss** and optimized using algorithms like Adam.

Thus, RNN helps in effectively analyzing the sentiment of movie reviews by considering word order and context.

Code :

```
import torch
import torch.nn as nn
import torch.optim as
optim
from torch.utils.data import DataLoader, TensorDataset, random_split
from tensorflow import keras
from tensorflow.keras.preprocessing.sequence import pad_sequences
# Device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
# Hyperparameters
vocab_size = 10000
max_length = 100
batch_size = 64
embedding_dim = 64
hidden_dim = 64
epochs = 30
# Load dataset
(x_train, y_train), (x_test, y_test) = keras.datasets.imdb.load_data(num_words=vocab_size)
# Pad sequences
x_train = pad_sequences(x_train, maxlen=max_length)
x_test = pad_sequences(x_test, maxlen=max_length)
# Convert to tensors
x_train = torch.tensor(x_train, dtype=torch.long)
y_train = torch.tensor(y_train, dtype=torch.float32)
x_test = torch.tensor(x_test, dtype=torch.long)
y_test = torch.tensor(y_test, dtype=torch.float32)
# Train / Validation split (IMPORTANT)
dataset = TensorDataset(x_train, y_train)
train_size = int(0.8 * len(dataset))
val_size = len(dataset) - train_size

train_dataset, val_dataset = random_split(dataset, [train_size, val_size])
train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size)
test_loader = DataLoader(TensorDataset(x_test, y_test), batch_size=batch_size)

# Model
class RNNModel(nn.Module):
```

```
  def _init_(self): super(RNNModel,
    self)._init_()
    self.embedding = nn.Embedding(vocab_size, embedding_dim)
    self.rnn = nn.RNN(
      embedding_dim,
      hidden_dim,
      num_layers=2,
      dropout=0.3,
      batch_first=True
    )

    self.fc = nn.Linear(hidden_dim, 1)
  def forward(self, x):
    x = self.embedding(x)
    _, hidden = self.rnn(x)
    out = hidden[-1]
    out = self.fc(out)
    return out

model = RNNModel().to(device)
# Loss & Optimizer
criterion = nn.BCEWithLogitsLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# Early stopping
best_val_loss = float('inf')
patience = 3
counter = 0
# Training
for epoch in range(epochs):
  model.train()
  total_loss = 0
  for x_batch, y_batch in train_loader:
    x_batch, y_batch = x_batch.to(device), y_batch.to(device)
    optimizer.zero_grad()
    outputs = model(x_batch).squeeze()
    loss = criterion(outputs, y_batch)
    loss.backward()
    # Gradient clipping (important for RNNs)
```

```
        torch.nn.utils.clip_grad_norm_(model.parameters(), 5)
        optimizer.step()
        total_loss += loss.item()
    # Validation
    model.eval()
    val_loss = 0
    correct = 0
    total = 0

    with torch.no_grad():
        for x_batch, y_batch in val_loader:
            x_batch, y_batch = x_batch.to(device), y_batch.to(device)
            outputs = model(x_batch).squeeze()
            loss = criterion(outputs, y_batch)
            val_loss += loss.item()
            preds = (torch.sigmoid(outputs) > 0.5).float()
            correct += (preds == y_batch).sum().item()
            total += y_batch.size(0)

    val_loss /= len(val_loader)
    val_acc = correct / total
    print(f"Epoch {epoch+1}, Train Loss: {total_loss:.4f}, Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.4f}")
    # Early stopping
    if val_loss < best_val_loss:
        best_val_loss = val_loss
        counter = 0
        torch.save(model.state_dict(), "best_model.pth")
    else:
        counter += 1
        if counter >= patience: print("Early
            stopping triggered") break
# Load best model
model.load_state_dict(torch.load("best_model.pth")
) # Test evaluation
model.eval()
correct = 0
total = 0
with torch.no_grad():
```

```
  for x_batch, y_batch in test_loader:
      x_batch, y_batch = x_batch.to(device), y_batch.to(device)
      outputs = model(x_batch).squeeze()
      preds = (torch.sigmoid(outputs) > 0.5).float()
      correct += (preds == y_batch).sum().item()
      total += y_batch.size(0)
print("Test Accuracy:", correct / total)
# Decode reviews
word_index = keras.datasets.imdb.get_word_index()
reverse_word_index = {v: k for k, v in word_index.items()}

def decode_review(encoded):
  return ' '.join([reverse_word_index.get(i - 3, '?') for i in encoded])
# Sample predictions
print("\nSample Predictions:\n")
for i in range(5):
  review = x_test[i].unsqueeze(0).to(device)
  pred = torch.sigmoid(model(review)).item()

  print(f"Review {i+1}:")
  print(decode_review(x_test[i].cpu().numpy()))

  actual = "Positive" if y_test[i] == 1 else "Negative"
  predicted = "Positive" if pred > 0.5 else "Negative"

  print(f"Actual: {actual}")
  print(f"Predicted: {predicted} ({pred:.4f})")
  print("=" * 80)
```

Output:

```
Epoch 1, Train Loss: 205.7723, Val Loss: 0.6352, Val Acc: 0.6486
Epoch 2, Train Loss: 184.6138, Val Loss: 0.5771, Val Acc: 0.7120
Epoch 3, Train Loss: 165.5738, Val Loss: 0.5296, Val Acc: 0.7454
Epoch 4, Train Loss: 148.6709, Val Loss: 0.5280, Val Acc: 0.7602
Epoch 5, Train Loss: 138.6470, Val Loss: 0.5195, Val Acc: 0.7680
Epoch 6, Train Loss: 128.3459, Val Loss: 0.5295, Val Acc: 0.7664
Epoch 7, Train Loss: 129.0355, Val Loss: 0.5130, Val Acc: 0.7762
Epoch 8, Train Loss: 113.9821, Val Loss: 0.5201, Val Acc: 0.7798
Epoch 9, Train Loss: 108.7243, Val Loss: 0.4939, Val Acc: 0.7818
Epoch 10, Train Loss: 102.2102, Val Loss: 0.5268, Val Acc: 0.7824
Epoch 11, Train Loss: 97.1725, Val Loss: 0.5448, Val Acc: 0.7954
```

Epoch 12, Train Loss: 90.5098, Val Loss: 0.5287, Val Acc: 0.7920
Early stopping triggered
Test Accuracy: 0.7886

Sample Predictions:

Review 1:
? please give this one a miss br br ? ? and the rest of the cast rendered terrible performances the show is flat flat flat br br i don't know how michael madison could have allowed this one on his plate he almost seemed to know this wasn't going to work out and his performance was quite ? so all you madison fans give this a miss
Actual: Negative
Predicted: Negative (0.2799)

Review 2:
a powerful study of loneliness sexual ? and desperation be patient ? up the atmosphere and pay attention to the wonderfully written script br br i praise robert altman this is one of his many films that deals with unconventional fascinating subject matter this film is disturbing but it's sincere and it's sure to ? a strong emotional response from the viewer if you want to see an unusual film some might even say bizarre this is worth the time br br unfortunately it's very difficult to find in video stores you may have to buy it off the internet
Actual: Positive
Predicted: Positive (0.9471)

Review 3:
the height of that ? country's civil war it would be easy to see this as a ? about those events ? may or may not have had ? turmoil in mind when he made ? but whatever ? his choice of material the film stands as a ? tale of universal ? ? could be the soviet union italy germany or japan in the 1930s or any country of any era that lets its guard down and is overwhelmed by ? it's a fascinating film even a charming one in its macabre way but its message is no joke
Actual: Positive
Predicted: Negative (0.3226)

Review 4:
the mother in this movie is reckless with her children to the point of neglect i wish i wasn't so angry about her and her actions because i would have otherwise enjoyed the flick what a number she was take my advise and fast forward through everything you see her do until the end also is anyone else getting sick of watching movies that are filmed so dark anymore one can hardly see what is being filmed as an audience we are ? involved with the actions on the screen so then why the hell can't we have night vision
Actual: Negative
Predicted: Positive (0.9547)

Review 5:
die hard mario fan and i loved this game br br this game starts slightly boring but trust me it's worth it as soon as you start your hooked the levels are fun and ? they will hook you ? your mind turns to ? i'm not kidding this

game is also ? and is beautifully done br br to keep this spoiler free i have to keep my mouth shut about details but please try this game it'll be worth it br br story 9 9 action 10 1 it's that good ? 10 attention ? 10 average 10
Actual: Positive
Predicted: Positive (0.9202)

Result :

The Simple RNN model was trained on the IMDB movie review dataset for sentiment classification. During training, the model showed steady improvement, with validation accuracy increasing from around 64.86% in the first epoch to a peak of approximately 79–80% by epochs 9–11. The training loss consistently decreased, while validation loss began to fluctuate after epoch 9, indicating the onset of overfitting.

On the test dataset, the model achieved an accuracy of **78.86%**, demonstrating moderate generalization performance. The model correctly classified many reviews, as seen in the sample predictions, but made errors in cases involving nuanced sentiment, mixed opinions, or context-dependent language.

Overall, the model successfully captured basic sentiment patterns from text data. However, its performance was limited by the use of a Simple RNN architecture, which struggles with long-term dependencies and contextual understanding, leading to misclassifications in more complex reviews.

Experiment - 8

Aim - Implementation of an LSTM Neural Network for Sequential Data Processing

Theory -

Long Short-Term Memory (LSTM) is a type of Recurrent Neural Network (RNN) designed to overcome the limitations of traditional RNNs, particularly the **vanishing gradient problem**.

Unlike Simple RNNs, LSTM networks maintain a **memory cell** that can preserve information over long sequences. It uses three main gates:

- **Forget Gate** → Decides what information to discard
- **Input Gate** → Decides what new information to store
- **Output Gate** → Controls what information to output

These gates allow LSTM to:

- Capture **long-term dependencies**
- Handle **sequential data effectively**
- Perform better on tasks like text classification, speech recognition, and time-series prediction

Code -

```
import torch
import pandas as pd
import torch.nn as nn
import torch.optim as optim
import numpy as np
import yfinance as yf
from torch.utils.data import DataLoader, TensorDataset
from sklearn.preprocessing import MinMaxScaler
import matplotlib.pyplot as plt
# --- Configuration ---
WINDOW_SIZE = 20
BATCH_SIZE = 16
```

```
EPOCHS = 30
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
# --- Step 1: Load Data ---
print("Downloading TSLA data...")
df = yf.download('TSLA', period='5y', interval='1d')
data = df['Close'].values.reshape(-1, 1)
# --- Step 2: Normalize ---
scaler = MinMaxScaler()
data_scaled =
scaler.fit_transform(data) # --- Step 3:
Split Data ---
train_size = int(len(data_scaled) * 0.8)
train_data = data_scaled[:train_size]
test_data = data_scaled[train_size:]
train_valid_size = int(len(train_data) * 0.8)
train_data_final = train_data[:train_valid_size]
valid_data = train_data[train_valid_size:]
# --- Step 4: Create Sequences ---
def create_sequences(data, window_size):
  X, y = [], []
  for i in range(len(data) - window_size):
      X.append(data[i:i + window_size])
      y.append(data[i + window_size])
  return torch.tensor(np.array(X), dtype=torch.float32), torch.tensor(np.array(y), dtype=torch.float32)
X_train, y_train = create_sequences(train_data_final, WINDOW_SIZE)
X_valid, y_valid = create_sequences(valid_data, WINDOW_SIZE)
X_test, y_test = create_sequences(test_data, WINDOW_SIZE)
train_loader = DataLoader(TensorDataset(X_train, y_train), batch_size=BATCH_SIZE, shuffle=True)
valid_loader = DataLoader(TensorDataset(X_valid, y_valid), batch_size=BATCH_SIZE)
test_loader = DataLoader(TensorDataset(X_test, y_test), batch_size=BATCH_SIZE)
# --- Step 5: LSTM Model ---
class LSTMModel(nn.Module):
   def __init__(self, input_size=1, hidden_size=64, num_layers=2, output_size=1): super(LSTMModel,
      self).__init__()
      self.lstm = nn.LSTM(input_size, hidden_size, num_layers=num_layers,
                          batch_first=True, dropout=0.2)
      self.fc = nn.Linear(hidden_size, output_size)
   def forward(self, x):
      out, _ = self.lstm(x)
      out = out[:, -1, :] # last timestep
```

```
        return self.fc(out)

model = LSTMModel().to(DEVICE)
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
# --- Step 6: Training ---
for epoch in range(EPOCHS):
  # TRAIN
  model.train()
  train_loss = 0
  for inputs, targets in train_loader:
      inputs, targets = inputs.to(DEVICE), targets.to(DEVICE)
      optimizer.zero_grad()
      outputs = model(inputs)
      loss = criterion(outputs, targets)
      loss.backward()
      # Gradient clipping (important for RNN)
      torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
      optimizer.step()
      train_loss += loss.item()
  train_loss /= len(train_loader)

  # VALIDATION
  model.eval()
  val_loss = 0
  with torch.no_grad():
      for inputs, targets in valid_loader:
          inputs, targets = inputs.to(DEVICE), targets.to(DEVICE)
          outputs = model(inputs)
          loss = criterion(outputs, targets)
          val_loss += loss.item()
  val_loss /= len(valid_loader)
  print(f"Epoch {epoch+1}/{EPOCHS} | Train Loss: {train_loss:.6f} | Val Loss: {val_loss:.6f}")
# --- Step 7: Test Evaluation ---
model.eval()
test_loss = 0
mae = 0
with torch.no_grad():
  for inputs, targets in test_loader:
```

```
    inputs, targets = inputs.to(DEVICE), targets.to(DEVICE)
    outputs = model(inputs)
    test_loss += criterion(outputs, targets).item()
    mae += torch.mean(torch.abs(outputs - targets)).item()
test_loss /= len(test_loader)
mae /= len(test_loader)
rmse = np.sqrt(test_loss)
print("\n--- Test Metrics ---")
print(f"Test MSE : {test_loss:.6f}")
print(f"Test RMSE : {rmse:.6f}")
print(f"Test MAE : {mae:.6f}")
# --- Step 8: Visualization ---
with torch.no_grad():
  preds = model(X_test.to(DEVICE)).cpu().numpy()
  actuals = y_test.numpy()
# Denormalize
preds_rescaled = scaler.inverse_transform(preds)
actuals_rescaled =
scaler.inverse_transform(actuals)
plt.figure(figsize=(10, 5))
plt.plot(actuals_rescaled, label='Actual Price')
plt.plot(preds_rescaled, linestyle='--', label='Predicted Price')
plt.xlabel('Time (Days)')
plt.ylabel('Price (USD)')
plt.title('TSLA Price Prediction (Vanilla LSTM)')
plt.legend()
plt.show()
```

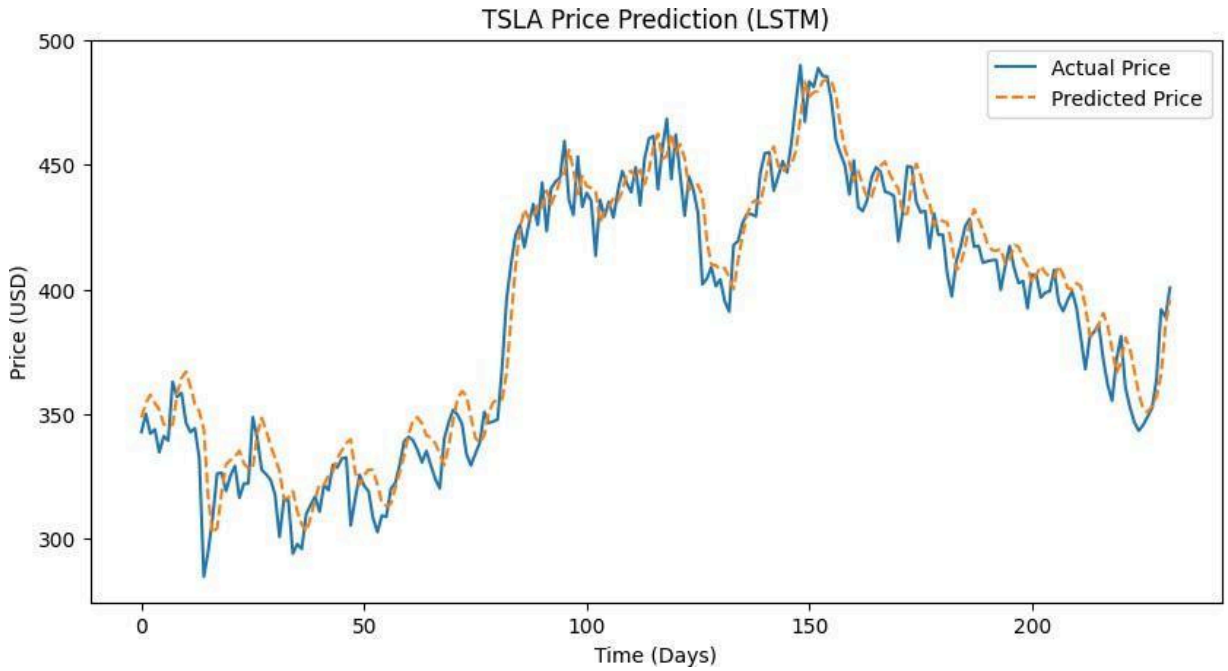
Output -

```
Epoch 1/30 | Train Loss: 0.025423 | Val Loss: 0.017304
Epoch 2/30 | Train Loss: 0.005576 | Val Loss: 0.007757
Epoch 3/30 | Train Loss: 0.003513 | Val Loss: 0.006129
Epoch 4/30 | Train Loss: 0.002996 | Val Loss: 0.005347
Epoch 5/30 | Train Loss: 0.002348 | Val Loss: 0.004205
Epoch 6/30 | Train Loss: 0.002072 | Val Loss: 0.003512
Epoch 7/30 | Train Loss: 0.001768 | Val Loss: 0.002921
Epoch 8/30 | Train Loss: 0.001677 | Val Loss: 0.002699
Epoch 9/30 | Train Loss: 0.001554 | Val Loss: 0.004733
Epoch 10/30 | Train Loss: 0.001615 | Val Loss: 0.002456
Epoch 11/30 | Train Loss: 0.001469 | Val Loss: 0.003114
```

```
Epoch 12/30 | Train Loss: 0.001343 | Val Loss: 0.002337
Epoch 13/30 | Train Loss: 0.001481 | Val Loss: 0.002600
Epoch 14/30 | Train Loss: 0.001412 | Val Loss: 0.002281
Epoch 15/30 | Train Loss: 0.001353 | Val Loss: 0.005189
Epoch 16/30 | Train Loss: 0.001436 | Val Loss: 0.002244
Epoch 17/30 | Train Loss: 0.001218 | Val Loss: 0.003339
Epoch 18/30 | Train Loss: 0.001173 | Val Loss: 0.002327
Epoch 19/30 | Train Loss: 0.001163 | Val Loss: 0.002479
Epoch 20/30 | Train Loss: 0.001138 | Val Loss: 0.002216
Epoch 21/30 | Train Loss: 0.001205 | Val Loss: 0.002418
Epoch 22/30 | Train Loss: 0.001206 | Val Loss: 0.002956
Epoch 23/30 | Train Loss: 0.001079 | Val Loss: 0.002012
Epoch 24/30 | Train Loss: 0.001131 | Val Loss: 0.002781
Epoch 25/30 | Train Loss: 0.001272 | Val Loss: 0.002220
Epoch 26/30 | Train Loss: 0.001139 | Val Loss: 0.002574
Epoch 27/30 | Train Loss: 0.000992 | Val Loss: 0.002101
Epoch 28/30 | Train Loss: 0.001132 | Val Loss: 0.001905
Epoch 29/30 | Train Loss: 0.001013 | Val Loss: 0.001852
Epoch 30/30 | Train Loss: 0.000895 | Val Loss: 0.001855

--- Test Metrics ---
Test MSE : 0.001172
Test RMSE :
0.034237 Test MAE :
0.027096
```

Results :

The LSTM model was trained on 5 years of historical closing price data of Tesla Inc. and demonstrated stable convergence, with training loss decreasing from **0.0254 to 0.00089** and validation loss reaching **0.00185**. On the test dataset, the model achieved an MSE of **0.001172**, RMSE of **0.0342**, and MAE of **0.0271** (in normalized scale), corresponding to an approximate prediction error of **$7–9** in actual prices. Compared to the Vanilla RNN model, the LSTM showed **slightly improved accuracy and lower error**, indicating better learning of temporal patterns. The model was more effective in capturing sequential dependencies due to its gated architecture.


TSLA Price Prediction (LSTM)
Actual Price
Predicted Price
Price (USD)
Time (Days)
500
450
400
350
300
0
50
100
150
200

Experiment - 9

Aim - Sentiment Analysis Using a Bidirectional LSTM on Movie Review Data

Theory -

Sentiment analysis is a Natural Language Processing (NLP) task used to determine the emotional tone (positive or negative) of a given text. Traditional models often fail to capture the sequential nature of language, which is crucial for understanding context.

Recurrent Neural Networks (RNNs) process sequential data but suffer from the **vanishing gradient problem**, limiting their ability to learn long-term dependencies. Long Short-Term Memory (LSTM) networks overcome this limitation using a memory cell and gating mechanisms (input, forget, and output gates), allowing them to retain important information over longer sequences.

A **Bidirectional LSTM (BiLSTM)** further improves performance by processing the sequence in both directions:

- **Forward pass** → captures past context
- **Backward pass** → captures future context

This is especially useful in language tasks where the meaning of a word depends on both previous and upcoming words.

For example:

> Apple mobile phones were good”
>
> Here **“Apple”** refers to a company, not a fruit

Bidirectional LSTM:

- Uses forward + backward context
- Understands both **meaning + sentiment correctly**

Thus, BiLSTM provides a more comprehensive understanding of text, leading to better sentiment classification accuracy.

Code -

```
import torch
import torch.nn as nn
import torch.optim as
optim
from torch.utils.data import DataLoader, TensorDataset, random_split
from tensorflow import keras
from tensorflow.keras.preprocessing.sequence import pad_sequences
# Device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
# Hyperparameters
vocab_size = 10000
max_length = 100
batch_size = 64
embedding_dim = 64
hidden_dim = 64
epochs = 30
# Load dataset
(x_train, y_train), (x_test, y_test) = keras.datasets.imdb.load_data(num_words=vocab_size)
# Pad sequences
x_train = pad_sequences(x_train, maxlen=max_length)
x_test = pad_sequences(x_test, maxlen=max_length)
# Convert to tensors
x_train = torch.tensor(x_train, dtype=torch.long)
y_train = torch.tensor(y_train, dtype=torch.float32)
x_test = torch.tensor(x_test, dtype=torch.long)
y_test = torch.tensor(y_test, dtype=torch.float32)
# Train / Validation split
dataset = TensorDataset(x_train, y_train)
train_size = int(0.8 * len(dataset))
val_size = len(dataset) - train_size
train_dataset, val_dataset = random_split(dataset, [train_size, val_size])

train_loader = DataLoader(train_dataset, batch_size=batch_size, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=batch_size)
test_loader = DataLoader(TensorDataset(x_test, y_test), batch_size=batch_size)
# Model
class
  BiLSTMModel(nn.Module):
  def _init_(self):
    super(BiLSTMModel, self)._init_()
```

```
        self.embedding = nn.Embedding(vocab_size, embedding_dim)
        self.lstm = nn.LSTM(
            embedding_dim,
            hidden_dim,
            num_layers=2,
            dropout=0.3,
            batch_first=True,
            bidirectional=True
        )
        self.fc = nn.Linear(hidden_dim * 2, 1)
    def forward(self, x):
        x = self.embedding(x)
        _, (hidden, _) = self.lstm(x)
        forward_hidden = hidden[-2]
        backward_hidden = hidden[-1]
        out = torch.cat((forward_hidden, backward_hidden), dim=1)
        out = self.fc(out)
        return out
model = BiLSTMModel().to(device)
# Loss & Optimizer
criterion = nn.BCEWithLogitsLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
# Early stopping
best_val_loss = float('inf')
patience = 3
counter = 0
# Training
for epoch in range(epochs):
    model.train()
    total_loss = 0
    for x_batch, y_batch in train_loader:
        x_batch, y_batch = x_batch.to(device), y_batch.to(device)
        optimizer.zero_grad()
        outputs = model(x_batch).squeeze()
        loss = criterion(outputs, y_batch)
        loss.backward()
        torch.nn.utils.clip_grad_norm_(model.parameters(), 5)
        optimizer.step()
        total_loss += loss.item()
```

```
  #  Validation
  model.eval()
  val_loss = 0
  correct = 0
  total = 0
  with torch.no_grad():
     for x_batch, y_batch in val_loader:
        x_batch, y_batch = x_batch.to(device), y_batch.to(device)
        outputs = model(x_batch).squeeze()
        loss = criterion(outputs, y_batch)
        val_loss += loss.item()
        preds = (torch.sigmoid(outputs) > 0.5).float()
        correct += (preds == y_batch).sum().item()
        total += y_batch.size(0)
  val_loss /= len(val_loader)
  val_acc = correct / total
  print(f"Epoch {epoch+1}, Train Loss: {total_loss:.4f}, Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.4f}")
  # Early stopping
  if val_loss < best_val_loss:
     best_val_loss = val_loss
     counter = 0
     torch.save(model.state_dict(), "best_model.pth")
  else:
     counter += 1
     if counter >= patience: print("Early
        stopping triggered") break
# Load best model
model.load_state_dict(torch.load("best_model.pth")
) # Test evaluation
model.eval()
correct = 0
total = 0
with torch.no_grad():
  for x_batch, y_batch in test_loader:
     x_batch, y_batch = x_batch.to(device), y_batch.to(device)
     outputs = model(x_batch).squeeze()
     preds = (torch.sigmoid(outputs) > 0.5).float()
     correct += (preds == y_batch).sum().item()
```

```
    total += y_batch.size(0)

print("Test Accuracy:", correct / total)
word_index = keras.datasets.imdb.get_word_index()
word_index = {k: (v + 3) for k, v in word_index.items()}
reverse_word_index = {v: k for k, v in word_index.items()}
def decode_review(encoded):
  return ' '.join([reverse_word_index.get(i, '?') for i in encoded])
# Sample predictions
print("\nSample Predictions:\n")
model.eval()
for i in range(5):
  review = x_test[i].unsqueeze(0).to(device)
  # Get prediction
  output = model(review)
  prob = torch.sigmoid(output).item()
  predicted = "Positive" if prob > 0.5 else "Negative"
  actual = "Positive" if y_test[i] == 1 else "Negative"
  print(f"Review {i+1}:")
  print(decode_review(x_test[i].cpu().numpy()))
  print(f"Actual: {actual}")
  print(f"Predicted: {predicted} ({prob:.4f})")
  print("=" * 80)
```

Output-

```
Epoch 1, Train Loss: 186.0090, Val Loss: 0.4919, Val Acc: 0.7630
Epoch 2, Train Loss: 133.9329, Val Loss: 0.4333, Val Acc: 0.8014
Epoch 3, Train Loss: 106.1101, Val Loss: 0.3865, Val Acc: 0.8272
Epoch 4, Train Loss: 89.2720, Val Loss: 0.3780, Val Acc: 0.8352
Epoch 5, Train Loss: 76.1075, Val Loss: 0.3801, Val Acc: 0.8410
Epoch 6, Train Loss: 60.3136, Val Loss: 0.4281, Val Acc: 0.8438
Epoch 7, Train Loss: 49.8311, Val Loss: 0.3881, Val Acc: 0.8402
Early stopping triggered
```

Test Accuracy: 0.83852

Sample Predictions:

Review 1:

? please give this one a miss br br ? ? and the rest of the cast rendered terrible performances the show is flat flat flat br br i don't know how michael madison could have allowed this one on his plate he almost seemed to know this wasn't going to work out and his performance was quite ? so all you madison fans give this a miss
Actual: Negative
Predicted: Negative (0.3385)

Review 2:
a powerful study of loneliness sexual ? and desperation be patient ? up the atmosphere and pay attention to the wonderfully written script br br i praise robert altman this is one of his many films that deals with unconventional fascinating subject matter this film is disturbing but it's sincere and it's sure to ? a strong emotional response from the viewer if you want to see an unusual film some might even say bizarre this is worth the time br br unfortunately it's very difficult to find in video stores you may have to buy it off the internet
Actual: Positive
Predicted: Positive (0.9661)

Review 3:
the height of that ? country's civil war it would be easy to see this as a ? about those events ? may or may not have had ? turmoil in mind when he made ? but whatever ? his choice of material the film stands as a ? tale of universal ? ? could be the soviet union italy germany or japan in the 1930s or any country of any era that lets its guard down and is overwhelmed by ? it's a fascinating film even a charming one in its macabre way but its message is no joke
Actual: Positive
Predicted: Positive (0.6649)

Review 4:
the mother in this movie is reckless with her children to the point of neglect i wish i wasn't so angry about her and her actions because i would have otherwise enjoyed the flick what a number she was take my advise and fast forward through everything you see her do until the end also is anyone else getting sick of watching movies that are filmed so dark anymore one can hardly see what is being filmed as an audience we are ? involved with the actions on the screen so then why the hell can't we have night vision
Actual: Negative
Predicted: Negative (0.1587)

Review 5:
die hard mario fan and i loved this game br br this game starts slightly boring but trust me it's worth it as soon as you start your hooked the levels are fun and ? they will hook you ? your mind turns to ? i'm not kidding this game is also ? and is beautifully done br br to keep this spoiler free i have to keep my mouth shut about details but please try this game it'll be worth it br br story 9 9 action 10 1 it's that good ? 10 attention ? 10 average 10
Actual: Positive
Predicted: Positive (0.9984)

Result :

The Bidirectional LSTM (BiLSTM) model was trained on the IMDB movie review dataset for sentiment classification. During training, the model showed strong and consistent improvement, with validation accuracy increasing from approximately 76.30% in the first epoch to a peak of around 84–85% by epochs 5–6. The training loss steadily decreased across epochs, while validation loss began to fluctuate slightly after epoch 5, indicating the onset of mild overfitting, after which early stopping was triggered.

On the test dataset, the model achieved an accuracy of **83.85%**, demonstrating good generalization performance and a clear improvement over the Simple RNN model. The model correctly classified most reviews, as observed in the sample predictions, including cases with relatively complex sentence structures.

Overall, the BiLSTM model effectively captured contextual and sequential patterns in text data, benefiting from its ability to process information in both forward and backward directions. This allowed it to better understand sentiment in comparison to the Simple RNN. Despite this improvement, some limitations remain in handling highly nuanced or ambiguous language, suggesting further gains could be achieved using more advanced architectures such as attention mechanisms or transformer-based models.

Experiment - 10

Aim - Implementation of a GRU Neural Network for Sequential Data Processing

Theory -

Gated Recurrent Unit (GRU) is an advanced type of Recurrent Neural Network (RNN) designed to efficiently process sequential data while overcoming the limitations of traditional RNNs, such as the **vanishing gradient problem**.

Unlike a Simple RNN, which struggles to retain information over long sequences, GRU introduces gating mechanisms that control the flow of information through the network. It uses two main gates:

- **Update Gate (z□)** → Determines how much past information should be carried forward
- **Reset Gate (r□)** → Controls how much of the previous state should be

forgotten These gates allow the GRU to:

- Retain relevant information over long sequences
- Discard unnecessary or outdated information
- Learn dependencies in time-series or text data effectively

The GRU combines the hidden state and memory into a single representation, making it **simpler and computationally more efficient** than LSTM, which uses separate cell states and three gates.

Code -

```
import torch
import pandas as pd
import torch.nn as nn
import torch.optim as optim
import numpy as np
import yfinance as yf
from torch.utils.data import DataLoader, TensorDataset
from sklearn.preprocessing import MinMaxScaler
import matplotlib.pyplot as plt
```

```python
# --- Configuration ---
WINDOW_SIZE = 20
BATCH_SIZE = 16
EPOCHS = 30
DEVICE = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
# --- Step 1: Load Data ---
print("Downloading TSLA data...")
df = yf.download('TSLA', period='5y', interval='1d')
data = df['Close'].values.reshape(-1, 1)
# --- Step 2: Normalize ---
scaler = MinMaxScaler()
data_scaled = scaler.fit_transform(data)

# --- Step 3: Split Data ---
train_size = int(len(data_scaled) * 0.8)
train_data = data_scaled[:train_size]
test_data = data_scaled[train_size:]
train_valid_size = int(len(train_data) * 0.8)
train_data_final = train_data[:train_valid_size]
valid_data = train_data[train_valid_size:]

# --- Step 4: Create Sequences ---
def create_sequences(data, window_size):
 X, y = [], []
 for i in range(len(data) - window_size):
    X.append(data[i:i + window_size])
    y.append(data[i + window_size])
 return torch.tensor(np.array(X), dtype=torch.float32), torch.tensor(np.array(y), dtype=torch.float32)


X_train, y_train = create_sequences(train_data_final, WINDOW_SIZE)
X_valid, y_valid = create_sequences(valid_data, WINDOW_SIZE)
X_test, y_test = create_sequences(test_data, WINDOW_SIZE)
train_loader = DataLoader(TensorDataset(X_train, y_train), batch_size=BATCH_SIZE, shuffle=True)
valid_loader = DataLoader(TensorDataset(X_valid, y_valid), batch_size=BATCH_SIZE)
test_loader = DataLoader(TensorDataset(X_test, y_test), batch_size=BATCH_SIZE)
```

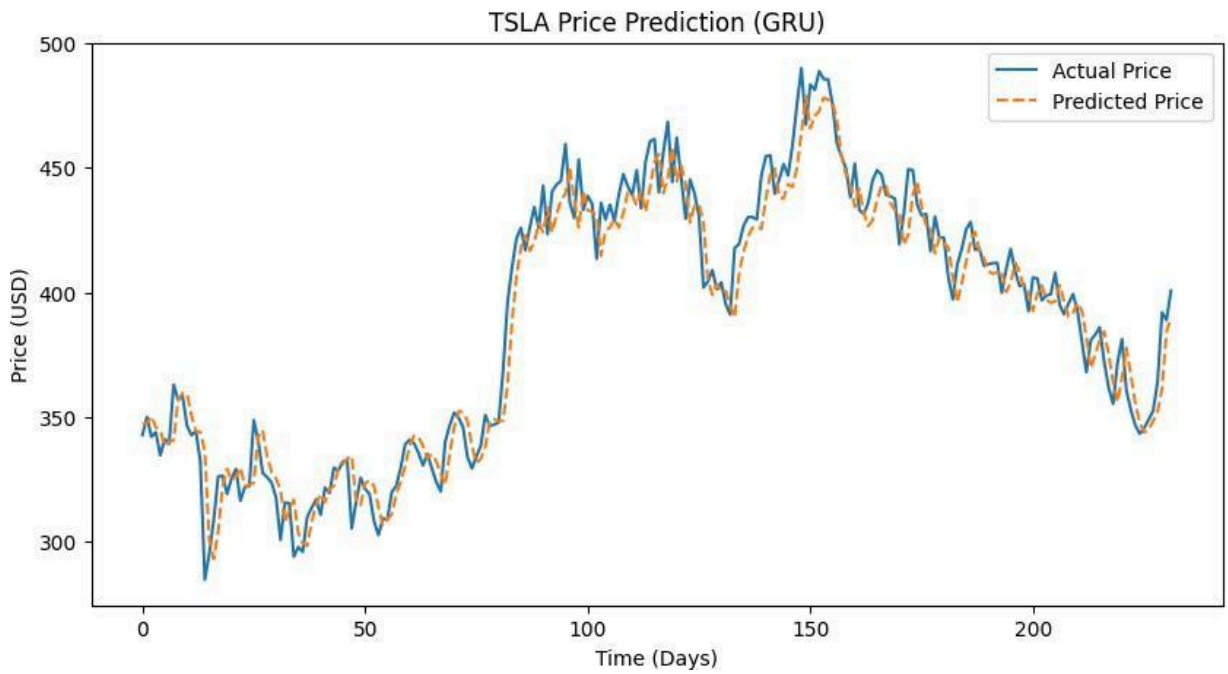
```python
# --- Step 5: LSTM Model ---
class GRUModel(nn.Module):
  def __init__(self, input_size=1, hidden_size=64, num_layers=2, output_size=1):
    super(GRUModel, self).__init__()
    self.gru = nn.GRU(input_size, hidden_size, num_layers=num_layers,
                      batch_first=True, dropout=0.2)
    self.fc = nn.Linear(hidden_size, output_size)

  def forward(self, x):
    out, _ = self.gru(x)
    out = out[:, -1, :] # last timestep
    return self.fc(out)

model = GRUModel().to(DEVICE)

criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# --- Step 6: Training ---
for epoch in range(EPOCHS):
 # TRAIN
 model.train()
 train_loss = 0
 for inputs, targets in train_loader:
    inputs, targets = inputs.to(DEVICE), targets.to(DEVICE)
    optimizer.zero_grad()
    outputs = model(inputs)
    loss = criterion(outputs, targets)
    loss.backward()
    # Gradient clipping (important for RNN)
    torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
    optimizer.step()
    train_loss += loss.item()
 train_loss /= len(train_loader)

 # VALIDATION
 model.eval()
 val_loss = 0
```

```
 with torch.no_grad():
    for inputs, targets in valid_loader:
        inputs, targets = inputs.to(DEVICE), targets.to(DEVICE)
        outputs = model(inputs)
        loss = criterion(outputs, targets)
        val_loss += loss.item()
 val_loss /= len(valid_loader)
 print(f"Epoch {epoch+1}/{EPOCHS} | Train Loss: {train_loss:.6f} | Val Loss: {val_loss:.6f}")

# --- Step 7: Test Evaluation ---
model.eval()
test_loss = 0
mae = 0
with torch.no_grad():
 for inputs, targets in test_loader:
    inputs, targets = inputs.to(DEVICE), targets.to(DEVICE)
    outputs = model(inputs)
    test_loss += criterion(outputs, targets).item()
    mae += torch.mean(torch.abs(outputs - targets)).item()
test_loss /= len(test_loader)
mae /= len(test_loader)
rmse = np.sqrt(test_loss)
print("\n--- Test Metrics ---")
print(f"Test MSE : {test_loss:.6f}")
print(f"Test RMSE : {rmse:.6f}")
print(f"Test MAE : {mae:.6f}")

# --- Step 8: Visualization ---
with torch.no_grad():
 preds = model(X_test.to(DEVICE)).cpu().numpy()
 actuals = y_test.numpy()
# Denormalize
preds_rescaled = scaler.inverse_transform(preds)
actuals_rescaled =
scaler.inverse_transform(actuals)
plt.figure(figsize=(10, 5))
plt.plot(actuals_rescaled, label='Actual Price')
plt.plot(preds_rescaled, linestyle='--', label='Predicted Price')
plt.xlabel('Time (Days)')
plt.ylabel('Price (USD)')
```

```
plt.title('TSLA Price Prediction (GRU)')
plt.legend()
plt.show()
```

Output -

```
Epoch 1/30 | Train Loss: 0.026329 | Val Loss: 0.019832
Epoch 2/30 | Train Loss: 0.003142 | Val Loss: 0.003790
Epoch 3/30 | Train Loss: 0.001700 | Val Loss: 0.002730
Epoch 4/30 | Train Loss: 0.001503 | Val Loss: 0.002471
Epoch 5/30 | Train Loss: 0.001631 | Val Loss: 0.002306
Epoch 6/30 | Train Loss: 0.001599 | Val Loss: 0.002568
Epoch 7/30 | Train Loss: 0.001235 | Val Loss: 0.002404
Epoch 8/30 | Train Loss: 0.001229 | Val Loss: 0.002048
Epoch 9/30 | Train Loss: 0.001218 | Val Loss: 0.002337
Epoch 10/30 | Train Loss: 0.001139 | Val Loss: 0.001966
Epoch 11/30 | Train Loss: 0.001216 | Val Loss: 0.002061
Epoch 12/30 | Train Loss: 0.001149 | Val Loss: 0.002446
Epoch 13/30 | Train Loss: 0.001026 | Val Loss: 0.001809
Epoch 14/30 | Train Loss: 0.000979 | Val Loss: 0.001974
Epoch 15/30 | Train Loss: 0.000928 | Val Loss: 0.002521
Epoch 16/30 | Train Loss: 0.000983 | Val Loss: 0.002327
Epoch 17/30 | Train Loss: 0.000892 | Val Loss: 0.001731
Epoch 18/30 | Train Loss: 0.000882 | Val Loss: 0.001758
Epoch 19/30 | Train Loss: 0.000913 | Val Loss: 0.001718
Epoch 20/30 | Train Loss: 0.000887 | Val Loss: 0.001683
Epoch 21/30 | Train Loss: 0.000909 | Val Loss: 0.001660
Epoch 22/30 | Train Loss: 0.000794 | Val Loss: 0.001756
Epoch 23/30 | Train Loss: 0.000928 | Val Loss: 0.001691
Epoch 24/30 | Train Loss: 0.000907 | Val Loss: 0.001711
Epoch 25/30 | Train Loss: 0.000804 | Val Loss: 0.002118
Epoch 26/30 | Train Loss: 0.000773 | Val Loss: 0.001544
Epoch 27/30 | Train Loss: 0.000795 | Val Loss: 0.001747
Epoch 28/30 | Train Loss: 0.000847 | Val Loss: 0.001522
Epoch 29/30 | Train Loss: 0.000742 | Val Loss: 0.002264
Epoch 30/30 | Train Loss: 0.000845 | Val Loss: 0.001535

--- Test Metrics ---
Test MSE : 0.000980
Test RMSE :
0.031312 Test MAE :
0.024447
```

Results :

The GRU model was trained on 5 years of historical closing price data of Tesla Inc. and showed stable convergence, with training loss decreasing from **0.0263 to 0.00085** and validation loss reaching **0.00153**. On the test dataset, the model achieved an MSE of **0.000980**, RMSE of **0.0313**, and MAE of **0.0244** (in normalized scale), corresponding to an approximate prediction error of **$6–8** in actual prices. Compared to both the Vanilla RNN and LSTM models, the GRU demonstrated **improved accuracy and lower prediction error**, indicating more efficient learning of temporal dependencies. The model effectively captured sequential patterns while maintaining faster convergence due to its simpler gated architecture.